

## Chapter 1: Introduction

### 1.1 Background

In today's digital era, technology has transformed many sectors such as banking, education, and governance, but traditional voting systems still rely on manual processes. These systems involve paper **ballots, physical polling stations, and large manpower, making them time-consuming, costly, and less efficient**. Additionally, traditional voting methods are prone to human errors, delays in result declaration, and risks of fraud or manipulation.

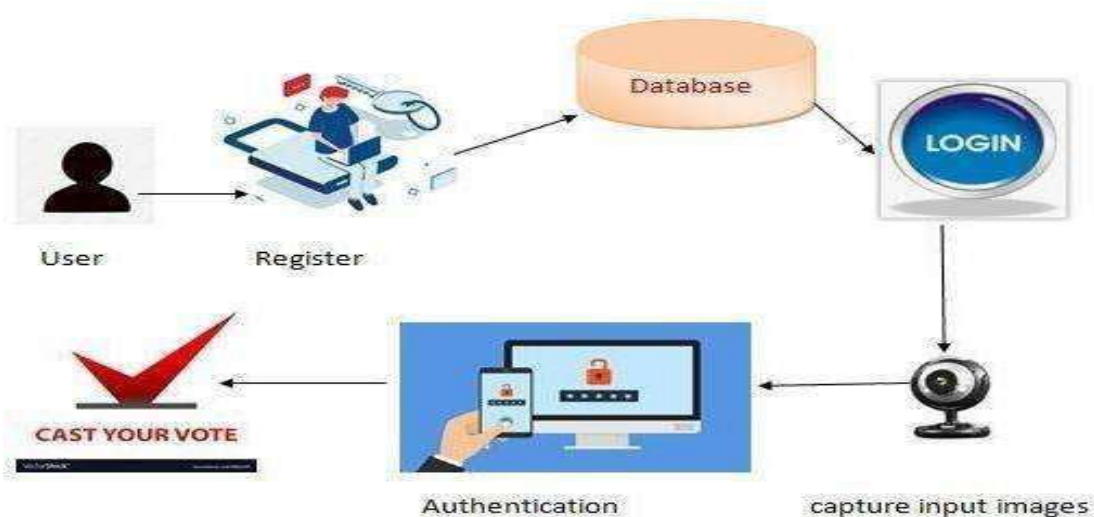
To overcome these challenges, an **Online Voting System** has been introduced as a modern solution. It is a web-based platform that allows voters to cast their votes securely from any location using the internet. This system improves **accessibility, reduces operational cost**, and ensures faster and more transparent election processes.

The system is developed using technologies like **HTML, CSS, and JavaScript** for the front-end interface, while backend technologies such as **Python (Flask)** are used to manage application logic. A **MySQL database** is used to store voter details, candidate information, and voting records securely.

Key features of the system include secure **user authentication**, which ensures that only authorized users can vote, and a one-person-one-vote mechanism to prevent multiple voting. The system also provides real-time vote counting and instant result generation, improving transparency and efficiency.

An admin panel is included to manage elections, candidates, and results effectively. The system is designed with a user-friendly interface, making it easy for users to register, log in, and vote without difficulty.

Security measures such as **password protection, data validation, and session management** are implemented to maintain data integrity and confidentiality



## 1.2 Objective

The main objective of the **Online Voting System** is to design and develop a secure, reliable, and efficient digital platform for conducting elections.

The system aims to overcome the limitations of traditional voting methods by providing a faster, more transparent, and user-friendly process.

In traditional voting systems, issues such as long queues, high operational costs, manual errors, and lack of transparency are common.

This project focuses on eliminating these problems by introducing an automated and online solution that simplifies the overall election process.

The key objectives of the system are as follows:

- To develop a **secure user authentication system** ensuring that only authorized voters can access the system.
- To implement a **one-person-one-vote mechanism** to avoid duplicate or fraudulent voting.
- To reduce the **time, cost, and manpower** required in traditional election processes.
- To provide **real-time vote counting and quick result generation** for better transparency.
- To design a **simple and user-friendly interface** for easy navigation and usability.
- To develop an efficient **admin module** for managing voters, candidates, and election activities.
- To ensure **data integrity, confidentiality, and security** using proper validation and protection techniques.
- To enable **remote accessibility**, allowing users to vote from any location using the internet.
- To minimize human intervention and reduce the chances of **errors and manipulation**.
- To create a scalable system that can be used for **small-scale as well as large-scale elections**.

### 1.3 Purpose & Scope

The purpose of the **Online Voting System** is to develop a secure, efficient, and user-friendly platform for conducting elections through an online medium. The system aims to replace traditional voting methods that are time-consuming, costly, and prone to errors. It provides a digital solution that allows voters to cast their votes easily and securely from any location.

**Key purposes of the system are:**

- To design a **secure and reliable voting platform** using modern web technologies.
- To provide **authenticated access** to users through login credentials.
- To ensure **accurate vote recording and automatic vote counting**.
- To reduce **manual work, paperwork, and overall election cost**.
- To implement **data security techniques** such as password encryption and session management.
- To handle **multiple users simultaneously** without performance issues.
- To maintain **data consistency and integrity** in the database.

The scope of the **Online Voting System** includes the development of a web-based application that can be used for conducting elections in organizations, institutions, and small-scale voting scenarios. The system provides features such as user registration, login authentication, candidate selection, vote casting, and result declaration.

The system is developed using modern web technologies such as **HTML, CSS, and JavaScript** for front-end design, and **Python (Flask)** for backend processing. A **MySQL database** is used to store and manage voter information, candidate details, and voting records efficiently.

It includes two main modules: the user module, which allows voters to participate in the election process, and the admin module, which enables the administrator to manage voters, candidates, and results efficiently.

The scope also includes implementing basic security features such as **login authentication, session handling, and vote restriction (one vote per user)**. The system can be deployed on a local server or web server, making it accessible through browsers on desktops or mobile devices.

The system focuses on providing basic security features such as authentication, data validation, and vote restriction to ensure fairness in the voting process. It also allows remote access, enabling users to vote from any location using the internet.

However, the scope is limited to web-based implementation and does not include advanced features like **biometric authentication**, blockchain security, or large-scale national election systems. These can be considered for future improvements.

## 1.4 Limitations

The **Online Voting System** provides a modern and efficient solution for conducting elections, it still has certain limitations. These limitations mainly arise due to **technical constraints, security concerns**, and dependency on internet connectivity. The system is designed for small to medium-scale use, and some advanced features are not included in the current implementation.

The system is mainly dependent on internet-based infrastructure, which can affect accessibility and performance. Additionally, since the application is developed using standard web technologies, it may face challenges in handling very large-scale elections with high user traffic.

The system does not support offline functionality, which means voting cannot be performed without internet access. This limits its usability in situations where connectivity is not available. Furthermore, proper data backup and recovery mechanisms are not fully implemented, which may result in data loss in case of system failure.

User-related challenges also play an important role. Some users may lack basic technical knowledge or digital literacy, making it difficult for them to use the system effectively.

**The limitations of the system are as follows:**

- The system is highly dependent on internet connectivity, so users without stable internet access may face difficulty in voting.
- There is a risk of **cyber security** threats such as **hacking, data breaches**, or unauthorized access if proper security measures are not implemented.
- Data stored in the database may be vulnerable if proper **encryption and backup mechanisms** are not implemented.
- The system currently uses basic authentication methods, which may not be as secure as advanced techniques like biometric or **OTP-based verification**.
- It may not be suitable for large-scale national elections due to scalability and high security requirements.
- Users with low technical knowledge or digital literacy may find it difficult to use the system.
- The system relies on server performance, and heavy traffic may cause delays or downtime.
- There is a possibility of system failure or technical errors, which can affect the voting process.
- The system does not include offline voting capability, so voting cannot be done without internet access.
- Advanced security features such as blockchain integration and end-to-end encryption are not implemented.

While the system improves efficiency and accessibility, these limitations must be considered and can be addressed in future enhancements to make the system more secure, scalable, and reliable.

## Chapter 2: Feasibility Study

### 2.1 Economic Feasibility

Economic Feasibility refers to the analysis of the cost and benefits associated with the development and implementation of the **Online Voting System**. It helps in determining whether the project is financially viable and whether the expected benefits outweigh the overall cost involved in the system.

In the case of the **Online Voting System**, the development cost is relatively low because it is based on widely available and open-source technologies such as **HTML, CSS, JavaScript, Python and MySQL**. These technologies do not require any licensing fees, which significantly reduces the overall cost of development.

Unlike traditional voting systems, which require expenses for printing ballot papers, setting up polling booths, transportation, security arrangements, and manpower, the online system minimizes these costs. The use of a digital platform eliminates the need for physical infrastructure and reduces the dependency on human resources.

The initial investment in this system mainly includes the cost of system development, server setup, and basic hardware requirements. Since the project can be developed using standard computers and free development tools, the initial setup cost remains affordable.

#### Cost factors involved in the system include:

- Development cost (coding, design, and testing)
- Hardware cost (computer systems, server setup)
- Maintenance cost (system updates and bug fixing)
- Internet and hosting cost
- Security implementation cost

#### Benefits of the system include:

- Reduction in paper and printing cost
- Decrease in manpower requirement
- Faster vote processing and result generation
- Lower operational and administrative cost
- Improved efficiency and accuracy

The Economic Feasibility analysis shows that the Online Voting System is financially viable, cost-effective, and beneficial for organizations and institutions looking to conduct elections in an efficient and economical manner.

## 2.2 Technical Feasibility

Technical Feasibility refers to the evaluation of the technical resources, tools, technologies, and expertise required to develop and implement the **Online Voting System**. It ensures that the proposed system can be developed using available technology and can operate efficiently without any major technical issues.

The Online Voting System is developed using modern and widely used web technologies. The front-end of the system is designed using **HTML, CSS, and JavaScript**, which provide an interactive and user-friendly interface. These technologies are easy to implement, well-supported, and compatible with all major web browsers.

For backend development, technologies such as **Python (Flask)** are used. These programming languages are powerful and efficient for handling server-side operations such as user authentication, vote processing, and database interaction. They also provide strong support for web application development and are suitable for building secure and scalable systems.

The system uses a **MySQL database** to store and manage all essential data, including user information, candidate details, and voting records. **MySQL** is a reliable and widely used relational database management system that ensures efficient data storage, retrieval, and management.

The system can be hosted on a local server (using tools like **XAMPP**) or deployed on a web server, making it accessible through web browsers on different devices such as desktops, laptops, and smartphones. This ensures platform independence and ease of access.

**Key technical aspects of the system include:**

- Use of **frontend technologies** (HTML, CSS, JavaScript) for designing user interface
- Use of **backend technologies** (Python/PHP) for application logic
- Integration of **MySQL database** for data storage and management
- Implementation of **authentication and authorization mechanisms**
- Use of **session management** to maintain user login states
- Validation of user inputs to prevent errors and unauthorized access

The system also supports **real-time data processing**, where votes are recorded and stored instantly in the database. Proper validation checks ensure that each user can vote only once, maintaining data integrity.

The **Online Voting System** is technically feasible as it uses standard, reliable, and easily available technologies. The system can be developed, deployed, and maintained with the available technical resources and knowledge, making it suitable for real-world applications.

## 2.3 Behavioral Feasibility

Behavioral Feasibility, also known as Operational or Human Feasibility, refers to the evaluation of how well the proposed system will be accepted and used by the end users. It focuses on user behavior, attitude, adaptability, and the overall interaction between the system and its users.

The Online Voting System, behavioral feasibility plays a crucial role because the success of the system depends on how easily users can adopt and trust the digital voting process. The system is designed with a **simple, user-friendly interface**, ensuring that users with basic computer knowledge can easily understand and operate it without any difficulty.

The system minimizes complexity by providing a **step-by-step process** for registration, login, candidate selection, and vote casting. This reduces confusion and ensures a smooth user experience. Proper validation messages and alerts are also included to guide users during the voting process.

The system does not require extensive training, as it is designed to be intuitive and easy to use. Basic guidance or instructions are sufficient for users to operate the system effectively. This makes it suitable for a wide range of users, including students and staff in organizations.

The system also considers **user satisfaction**, ensuring that the voting process is quick, reliable, and free from errors. Fast response time, clear navigation, and simple design contribute to a positive user experience.

### Key behavioral aspects of the system include:

- Easy-to-use and **intuitive user interface**
- Minimal training requirement for users
- High level of **user interaction and guidance**
- Improved **user satisfaction and convenience**
- Support for users with basic technical knowledge
- Measures to build **trust and confidence** in the system

As more users become familiar with digital platforms, the adoption rate of online voting systems is expected to increase.

The Online Voting System is behaviorally feasible as it is designed to be user-friendly, easy to adopt, and capable of gaining user trust with proper awareness and minimal training.

## Chapter 3 Requirement & Analysis

### 3.1 Problem Definition

The voting system plays a crucial role in decision-making processes in organizations, institutions, and governments. However, traditional voting methods face several issues related to efficiency, security, and accessibility. This project identifies and addresses these problems by proposing a web-based Online Voting System.

#### Problems in Existing System

##### 1. Time Consuming Process

- Manual voting requires voters to stand in long queues
- Vote counting takes several hours or even days

##### 2. High Operational Cost

- Cost of printing ballot papers
- Setup of polling stations
- Requirement of manpower and security

##### 3. Limited Accessibility

- Voters must be physically present at polling stations
- Difficult for remote area voters

##### 4. Security Issues

- Chances of fake or duplicate voting
- Unauthorized access to voting system
- Possibility of vote tampering

##### 5. Lack of Transparency

- Manual counting may lead to errors
- Difficult to verify fairness of election
- Trust issues among voters

##### 6. Human Errors

- Mistakes during vote counting
- Data entry errors
- Mismanagement by staff

##### 7. Inefficient Data Management

- Maintaining records manually is difficult
- Risk of data loss or damage

## 3.2 Planning & Scheduling (Flow Chart & Gantt Chart)

### 3.2.1 Project Planning

Planning and scheduling are essential parts of software development. They help in organizing tasks, managing time effectively, and ensuring that the project is completed within the given deadline.

In the development of the **Online Voting System**, proper planning is required to define the sequence of activities, allocate resources, and track progress.

#### 1. Requirement Gathering

- Collecting system requirements from users
- Understanding features like login, voting, admin panel

#### 2. System Analysis

- Analyzing feasibility of the system
- Identifying hardware and software needs
- Studying current voting problems

#### 3. System Design

- Designing database (MySQL)
- Creating UI design using HTML, CSS
- Planning backend logic using Flask

#### 4. Development

- Frontend development (HTML, CSS, JS)
- Backend development (Python Flask)
- Database integration

#### 5. Testing

- Unit testing
- Integration testing
- Bug fixing

#### 6. Deployment

- Hosting the application
- Final system setup

#### 7. Maintenance

- Fixing bugs after deployment
- Updating features
- Improving performance

### 3.2.2 Project Scheduling

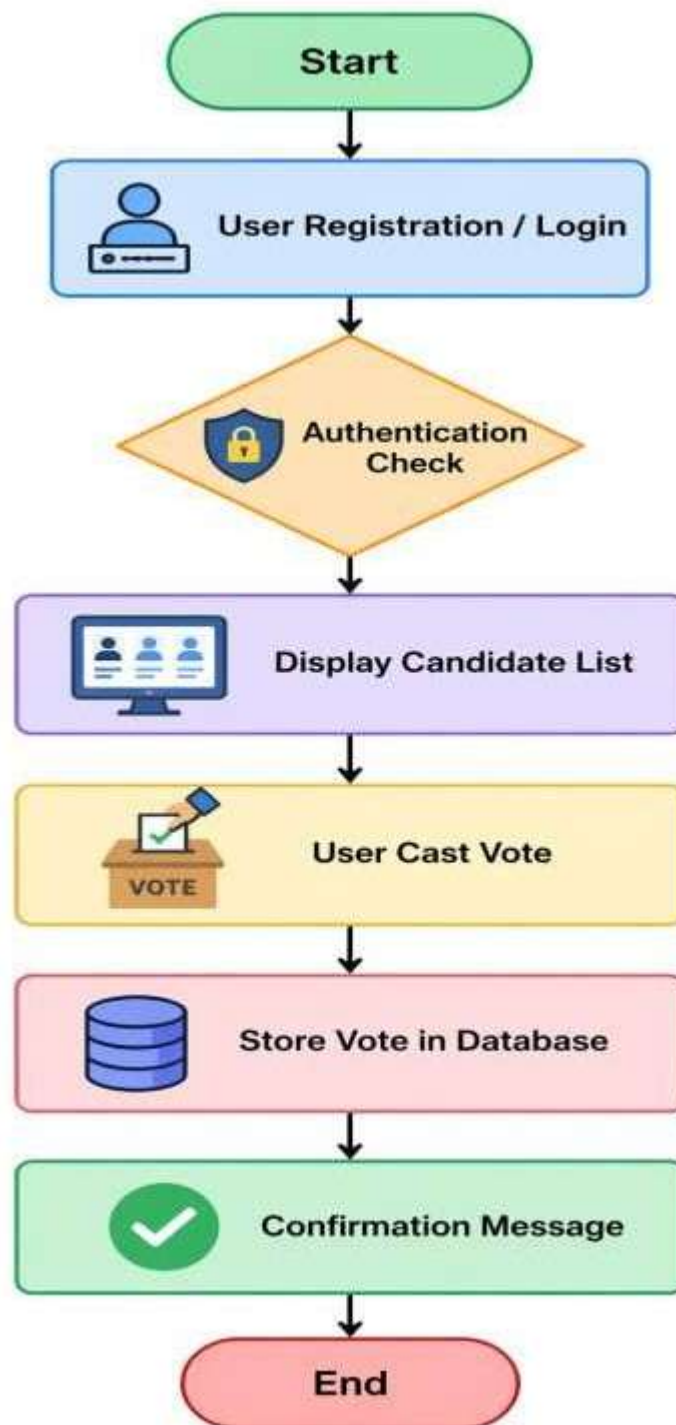
Project scheduling defines the timeline for completing each phase of the project. It ensures that all tasks are completed within a fixed time frame.

Scheduling helps in dividing the project into different phases and assigning time durations to each task.

#### Gantt Chart for Online Voting System

| Phase                 | Duration | Task                                     | Start Date | End Date | Dependency                       |
|-----------------------|----------|------------------------------------------|------------|----------|----------------------------------|
| Requirement Gathering | 3 days   | Planning, Feasibility Study              | Week 1     | Week 1   | Requirement Gathering            |
| System Analysis       | 4 Days   | Requirement Gathering & Analysis         | Week 2     | Week 2   | Planning                         |
| System Design         | 5 Days   | ER Diagram & Data Modeling               | Week 3     | Week 3   | System Design                    |
| Development           | 10 Days  | Frontend, Backend & Database Development | Week 4     | Week 5   | Design, Frontend Dev, ER Diagram |
| Testing               | 4 Days   | Unit Testing, System Testing, Bug Fixing | Week 6     | Week 6   | Unit Testing, Testing            |
| Deployment            | 2 Days   | Deployment & Hosting                     | Week 7     | Week 7   | Final Testing & Deployment       |

## Flow Chart of Online Voting Process



### 3.3 Software & Hardware Requirements

#### 3.3.1 Software Requirements

Every software system requires a proper set of software and hardware components for its development and execution. The Online Voting System is a web-based application that uses modern technologies such as HTML, CSS, JavaScript, Python (Flask), and MySQL.

##### 1. Operating System

- Window 11/10

##### 2. Frontend Technologies

###### HTML (Hyper Text Markup Language)

- Used for designing structure of web pages
- Creates forms (login, registration, voting page)

###### CSS (Cascading Style Sheets)

- Used for styling web pages
- Makes UI attractive and responsive

###### JavaScript

- Adds interactivity
- Used for form validation
- Enhances user experience

##### 3. Backend Technology (Python/Flask)

- Core programming language
- Handles server-side logic
- Lightweight web framework
- Used to connect frontend with backend
- Handles routing and APIs

##### 4. Database (MySQL)

- Used for storing data
- Stores user details, candidates, votes
- Ensures data consistency

##### 5. Development Tools

- VS code (Code editor)

##### 6. Other Software Requirement

- Git (Version control)

#### 3.3.2 Hardware Requirements

##### 1. Minimum Hardware Requirements

- **Processor:** RYZEN 5
- **RAM:** 4 GB
- **Hard Disk:** 500 GB
- **Internet Connection:** Required

## 2. Recommended Hardware Requirements

- **Processor:** Intel i5
- **RAM:** 8 GB
- **Storage:** SSD for better performance
- **Network:** High – speed internet

## 3. Server Requirements

- Web server (localhost)
- Database server (MySQL)
- Secure hosting environment

### 3.4 SDLC Model (Software Development Life Cycle)

The Software Development Life Cycle (SDLC) is a structured process used for developing high-quality software in a systematic and organized manner. It divides the entire development process into different phases, ensuring proper planning, execution, and maintenance of the system.

In the context of the **Online Voting System**, SDLC plays a crucial role in organizing the development process. Since the system involves sensitive operations such as user authentication and vote management, following a proper development model is essential to ensure reliability and security.

For this project, the **Waterfall Model** is chosen because of its simplicity, clarity, and sequential execution. Each phase is completed before moving to the next, making it easy to manage and document.

#### Phases of SDLC:

##### 1. Requirement Analysis

This is the first and most important phase of SDLC. In this stage, all the system requirements are gathered from stakeholders and analyzed carefully.

- Understanding user requirements (voters and admin)
- Identifying system features such as registration, login, vote casting, and result viewing
- Defining system constraints and limitations
- Preparing Software Requirement Specification (SRS) document

##### 2. System Design

In this phase, the system architecture and design are created based on the requirements gathered earlier.

- Designing database schema (tables like Users, Candidates, Votes)
- Creating user interface layout (forms, dashboards)

- Defining system modules (admin module, user module)
- Preparing DFD (Data Flow Diagram) and ER Diagram

### 3. Implementation

This phase involves converting the design into actual code.

#### Technologies Used:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python (Flask)
- **Database:** MySQL

#### Activities Performed:

- Developing user interfaces (login, voting page)
- Writing backend logic for authentication and vote processing
- Connecting frontend with backend
- Integrating database

### 4. Testing

Testing ensures that the system works correctly and is free from errors.

#### Types of Testing:

- **Unit Testing:** Testing individual components
- **Integration Testing:** Checking interaction between modules
- **System Testing:** Testing the complete system
- **Security Testing:** Ensuring data safety

#### Activities Performed:

- Identifying bugs and errors
- Fixing issues
- Verifying system functionality

### 5. Deployment

In this phase, the developed system is released for actual use.

#### Activities Performed:

- Hosting the application on a server
- Configuring database and environment
- Making system accessible to users

#### Deployment Types:

- Localhost
- Cloud Server

## 6. Maintenance

After deployment, the system enters the maintenance phase.

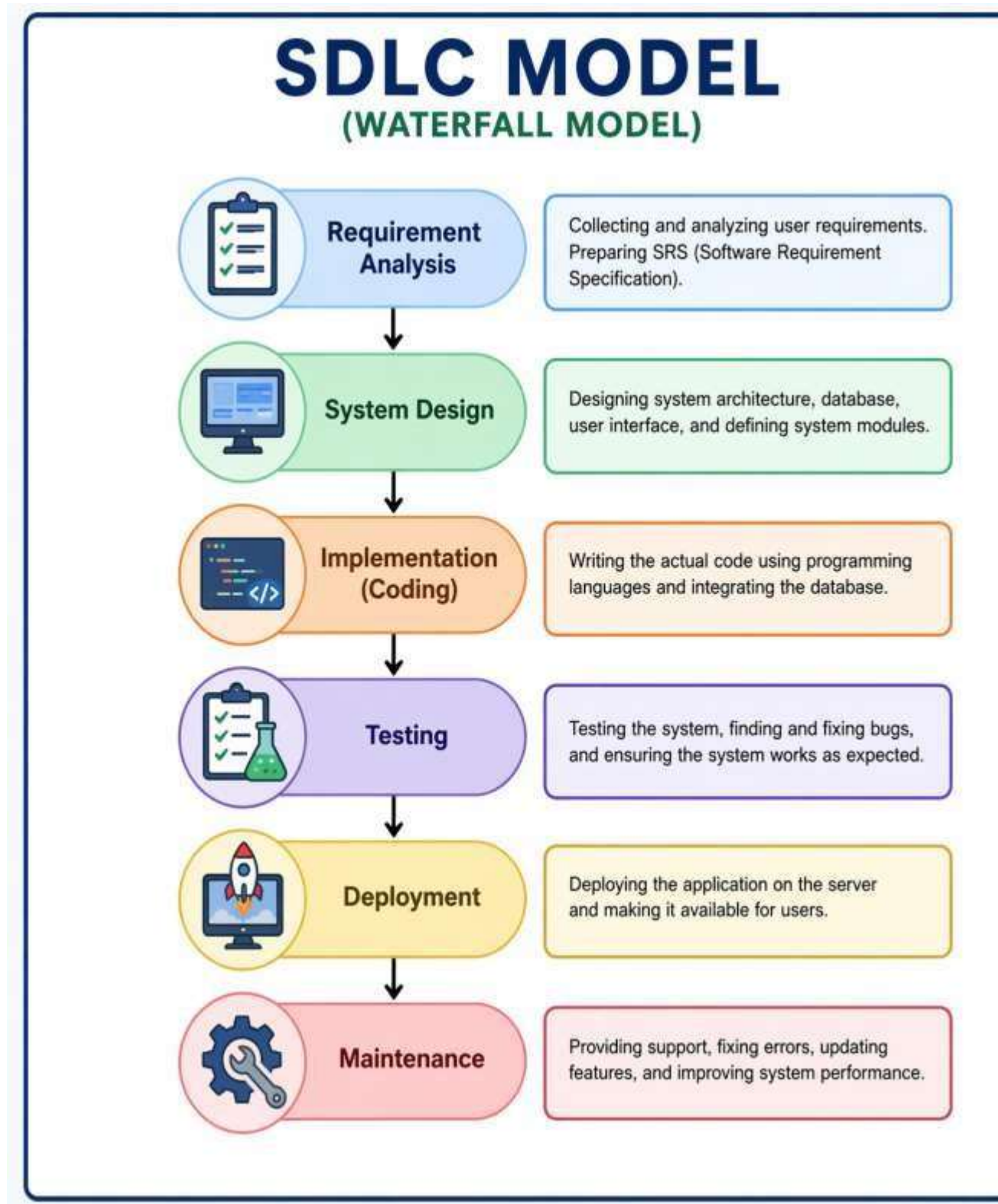
### Activities Performed:

- Fixing bugs reported by users
- Updating system features
- Improving performance and security
- Regular monitoring

### Types of Maintenance:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance

## SDLC Model Diagram:



## Chapter 4: Survey Of Technology

### 4.1 Technology Used (Front End, Back End & Database)

The Online Voting System is developed using modern web technologies that provide a secure, scalable, and efficient platform. The system follows a **three-tier architecture**, which separates the user interface, application logic, and data storage. This separation improves maintainability, flexibility, and performance.

The technologies used in this project include **HTML, CSS, JavaScript (Frontend), Python with Flask (Backend), and MySQL (Database)**. Each technology plays a specific role in building the complete system.

#### 4.1.1 Frontend Technologies

Frontend technologies are responsible for designing the user interface and improving user experience.

##### HTML (Hyper Text Markup Language)

HTML is the backbone of any web application. It is used to create the structure and layout of web pages.

##### Key Features:

- Simple and easy to use
- Platform-independent
- Supports multimedia elements
- Provides form handling capabilities

##### Role in Online Voting System:

- Designing login and registration forms
- Creating voting interface
- Structuring admin dashboard
- Displaying candidate lists and results



```
<body>
<nav class="navbar navbar-expand-lg navbar-dark fixed-top">
  <div class="container">
    <a class="navbar-brand fw-bold"
      href="{{ url_for('dashboard') if session.get('user_id') else url_for('index') }}">
    <i class="fas fa-vote-yea me-2"></i>
    <span>VOTE</span><span class="text-warning">SECURE</span>
    </a>
    <button class="navbar-toggler border-0" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="navbarNav">
      <!-- Left Navigation -->
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link" href="#home" data-bs-toggle="modal" data-bs-target="#homeModal">
            <i class="fas fa-home"></i> Home
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#about" data-bs-toggle="modal" data-bs-target="#aboutModal">
            <i class="fas fa-info-circle"></i> About
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#contact" data-bs-toggle="modal" data-bs-target="#contactModal">
            <i class="fas fa-envelope"></i> Contact
          </a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

## CSS (Cascading Style Sheets)

CSS is used to style and enhance the appearance of web pages.

### Key Features:

- Controls layout and design
- Supports responsive design
- Enables animations and transitions

### Role in Project:

- Designing attractive UI
- Styling buttons, forms, and tables

```
<!doctype html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Online Voting System{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.2/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
    rel="stylesheet">
  <style>
    :root {
      --primary: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      --secondary: linear-gradient(135deg, #ff093b 0%, #ff576e 100%);
      --success: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
    }

    * {
      font-family: 'Poppins', sans-serif;
    }

    body {
      background: linear-gradient(135deg, #f5f7fa 0%, #e3cfe2 100%);
      min-height: 100vh;
    }
  </style>
</head>
```

## JavaScript

JavaScript is a scripting language used to add dynamic behavior to web pages.

### Key Features:

- Client-side scripting
- Event handling
- Real-time interaction

### Role in Project:

- Form validation (username, password)
- Preventing invalid inputs
- Displaying alerts and notifications
- Enhancing user interaction

### Advantages:

- Fast execution
- Improves responsiveness
- Reduces server load

```
// Vote Form Handler
document.addEventListener('DOMContentLoaded', function () {--
});

// Alert Function
function showAlert(message, type = 'info') {
  const alertDiv = document.createElement('div');
  alertDiv.className = `alert alert-${type} alert-dismissible fade show position-fixed`;
  alertDiv.style.cssText = `top: 20px; right: 20px; z-index: 9999; min-width: 300px;`;
  alertDiv.innerHTML = `
    ${message}
    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
  `;

  document.body.appendChild(alertDiv);

  // Auto remove after 5 seconds
  setTimeout(() => {
    if (alertDiv.parentNode) {
      alertDiv.remove();
    }
  }, 5000);
}

// Chart.js for Results (if needed)
function renderResultsChart(canvasId, data) {
  const ctx = document.getElementById(canvasId).getContext('2d');
  new Chart(ctx, {
```

#### 4.1.2. Backend Technology

Backend technologies handle server-side logic and communication with the database.

##### Python

Python is a powerful and easy-to-learn programming language used for backend development.

##### Key Features:

- Simple syntax
- High readability
- Large community support
- Cross-platform

##### Role in Project:

- Processing user requests
- Managing voting logic
- Handling authentication
- Interacting with database

##### Advantages:

- Rapid development
- Easy debugging
- Scalable applications

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify
from flask_mysql import MySQL
from werkzeug.security import generate_password_hash, check_password_hash
from config import Config
import MySQLdb.cursors
import re

app = Flask(__name__)
app.config.from_object(Config)
mysql = MySQL(app)

# ALL ROUTES IN PROPER ORDER
@app.route('/')
def index():
    return render_template('login.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username'].strip()
        email = request.form['email'].strip().lower()
        password = generate_password_hash(request.form['password'])

        cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)

        cursor.execute("SELECT id FROM users WHERE email = %s", (email,))
        if cursor.fetchone():
            cursor.close()
```

**Flask Framework**

Flask is a lightweight web framework written in Python.

**Key Features:**

- Minimalistic and flexible
- Built-in development server
- RESTful request handling

**Role in Project:**

- Routing URLs (login, vote, result)
- Connecting frontend with backend
- Managing sessions and cookies

**Advantages:**

- Easy to learn
- Fast development
- Highly customizable

**4.1.3 Database Technology****MySQL**

MySQL is a relational database management system (RDBMS) used for storing structured data.

**Key Features:**

- High performance
- Secure data handling
- Supports SQL queries

**Role in Project:**

- Storing user details
- Managing candidate information
- Recording votes securely

**Tables Used:**

- Users Table
- Candidates Table
- Votes Table

**Advantages:**

- Reliable and efficient
- Easy data retrieval
- Scalable for large data

```

CREATE DATABASE voting_system;

USE voting_system;

-- Users Table
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('voter', 'admin') DEFAULT 'voter',
  has_voted BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Candidates Table
CREATE TABLE candidates (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  party VARCHAR(50),
  photo VARCHAR(255),
  position VARCHAR(50),
  votes INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Votes Table
CREATE TABLE votes (

```

## Technology Comparison

| Technology | Purpose     | Advantage        |
|------------|-------------|------------------|
| HTML       | Structure   | Easy to use      |
| CSS        | Design      | Responsive UI    |
| JavaScript | Interaction | Dynamic behavior |
| Python     | Backend     | Fast Development |
| Flask      | Framework   | Lightweight      |
| MySQL      | Database    | Secure Storage   |

## 4.2 IDE & Various Tools

The development of the Online Voting System requires various Integrated Development Environments (IDEs) and supporting tools to ensure efficient coding, testing, debugging, and deployment. These tools help developers to write clean code, manage databases, test APIs, and maintain version control.

The selection of appropriate tools improves productivity, reduces errors, and ensures smooth project development.

### 1. Visual Studio Code (VS Code)

Visual Studio Code is a lightweight and powerful source code editor developed by Microsoft.

#### Key Features:

- Supports multiple programming languages
- Built-in terminal
- Syntax highlighting and auto-completion
- Extension support (Python, Flask, Live Server)

#### Usage in Project:

- Writing frontend code (HTML, CSS, JavaScript)
- Writing backend code (Python Flask)
- Debugging and running the application

#### Advantages:

- Fast and lightweight
- Highly customizable
- Free and open-source

### 2. PyCharm

PyCharm is a professional IDE specifically designed for Python development.

#### Key Features:

- Smart code completion
- Built-in debugging tools
- Integrated terminal
- Framework support (Flask, Django)

#### Usage in Project:

- Writing and managing backend code
- Debugging Python scripts
- Managing project files

#### Advantages:

- Improves coding efficiency
- Helps detect errors quickly
- Ideal for Python-based projects

### 3. MySQL Workbench

MySQL Workbench is a graphical tool used for database design and management.

**Key Features:**

- Database modeling
- SQL query execution
- Data visualization
- Backup and recovery

**Usage in Project:**

- Creating database schema
- Managing tables (Users, Candidates, Votes)
- Running SQL queries

**Advantages:**

- Easy database management
- User-friendly interface
- Reduces manual SQL errors

### 4. Git (Version Control System)

Git is a distributed version control system used to track changes in code.

**Key Features:**

- Tracks code changes
- Supports branching and merging
- Enables collaboration

**Usage in Project:**

- Managing project versions
- Tracking changes in code
- Backup of source code

**Advantages:**

- Prevents data loss
- Easy rollback to previous versions
- Useful for teamwork

## 5. Web Browsers (Google Chrome / Edge)

Web browsers are used to run and test the web application.

### Key Features:

- Fast rendering
- Developer tools (Inspect Element)
- Debugging JavaScript

### Usage in Project:

- Testing frontend UI
- Debugging HTML/CSS/JS
- Checking responsiveness

### Advantages:

- Real-time testing
- Easy debugging
- Cross-platform support

## 6. XAMPP / Local Server (Optional Tool)

XAMPP is used to create a local server environment.

### Key Features:

- Apache server
- MySQL database
- Local hosting

### Usage in Project:

- Running project locally
- Testing database connectivity

## Chapter 5: Preliminary Module Description

### Preliminary Module Description

The Online Voting System is divided into different modules to ensure proper organization, easy development, and efficient functioning. Each module performs a specific task and interacts with other modules to complete the overall system process.

The modular approach helps in simplifying system design, improving maintainability, and reducing complexity.

### Main Modules of the System

The system is divided into the following modules:

1. User Module
2. Admin Module
3. Authentication Module
4. Voting Module
5. Result Module
6. Database Module

#### User Module

The User Module handles all activities related to voters.

##### Functions:

- User registration
- User login
- Viewing candidate list
- Casting vote
- Viewing results (if allowed)

##### Features:

- Simple interface
- Secure access
- One-time voting

### 2. Admin Module

The Admin Module is responsible for managing the entire system.

##### Functions:

- Admin login
- Add/update/delete candidates
- Manage users
- Start/stop voting process
- View results

##### Features:

- Full control over system
- Secure admin panel
- Real-time monitoring

### 3. Authentication Module

This module ensures that only authorized users can access the system.

**Functions:**

- Login verification
- Password validation
- Session management

**Features:**

- Prevents unauthorized access
- Ensures one user = one vote
- Secure login system

### 4. Voting Module

This is the core module of the system where voting takes place.

**Functions:**

- Display candidate list
- Allow vote selection
- Submit vote

**Features:**

- One-time vote submission
- Secure vote storage
- Easy user interface

### 5. Result Module

This module calculates and displays election results.

**Functions:**

- Count votes
- Generate results
- Display winner

**Features:**

- Fast result generation
- Accurate counting
- Real-time updates

### 6. Database Module

This module manages all data in the system.

**Functions:**

- Store user data
- Store candidate data
- Store votes

**Features:**

- Secure data storage
- Efficient data retrieval
- Data consistency

## Chapter 6: System Design

### 6.1 Data Flow diagram

A Data Flow Diagram (DFD) is a graphical tool used to represent the flow of data within a system. It describes how data enters the system, how it is processed, where it is stored, and how it leaves the system.

In the **Online Voting System**, the DFD plays a crucial role in understanding the interaction between users, admin, and the system. It helps in identifying various processes such as user authentication, vote casting, data storage, and result generation.

DFD is widely used in system design because it provides a clear and simple visualization of complex processes.

#### Basic Components of DFD

##### 1. Process

A process represents an activity that transforms input data into output.

##### Examples in this system:

- User Login
- Vote Casting
- Result Generation

##### 2. Data Flow

Data flow shows the movement of data from one part of the system to another.

##### Examples:

- Login credentials
- Vote data
- Result data

##### 3. Data Store

A data store represents a place where data is stored.

##### Examples:

- User Database
- Candidate Database
- Votes Database

##### 4. External Entity

External entities are users or systems that interact with the system.

##### Examples:

- **Voter (User)**
- **Admin**

### Rules for Drawing DFD

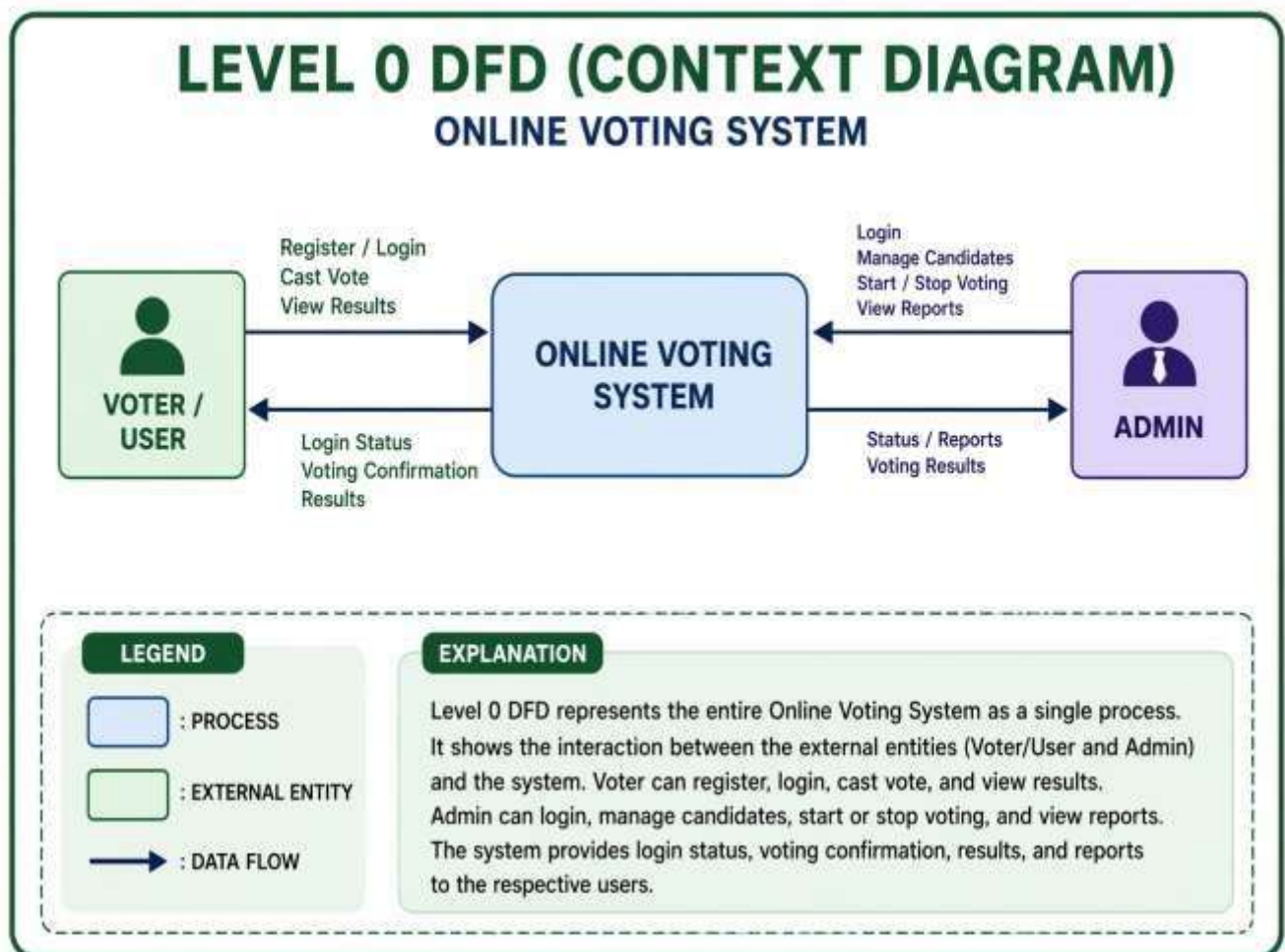
- Every process must have at least one input and one output
- Data cannot move directly between external entities without a process
- Data must pass through a process before entering a data store
- Use clear and meaningful names for processes
- Avoid complex diagrams; keep them simple and readable

### Levels of DFD

DFD is divided into multiple levels to show increasing detail.

### Level 0 DFD (Diagram)

This is the highest level of DFD where the entire system is represented as a single process.

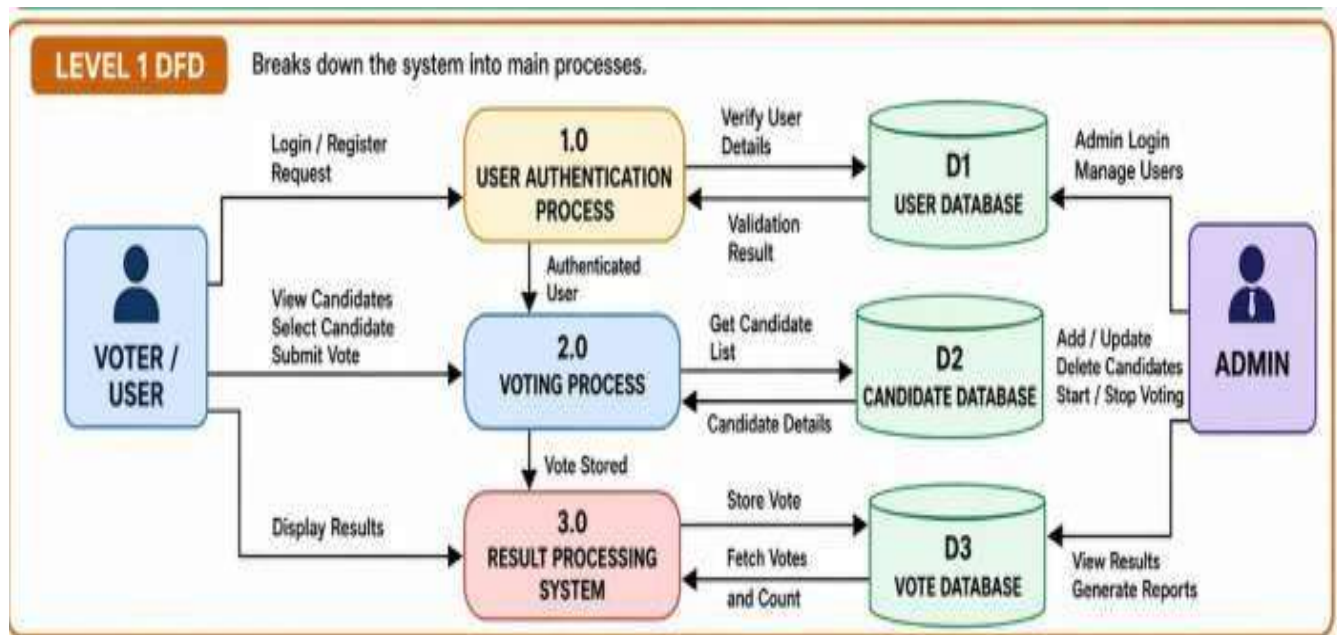


**Level 1 DFD**

Level 1 DFD breaks the system into main functional processes.

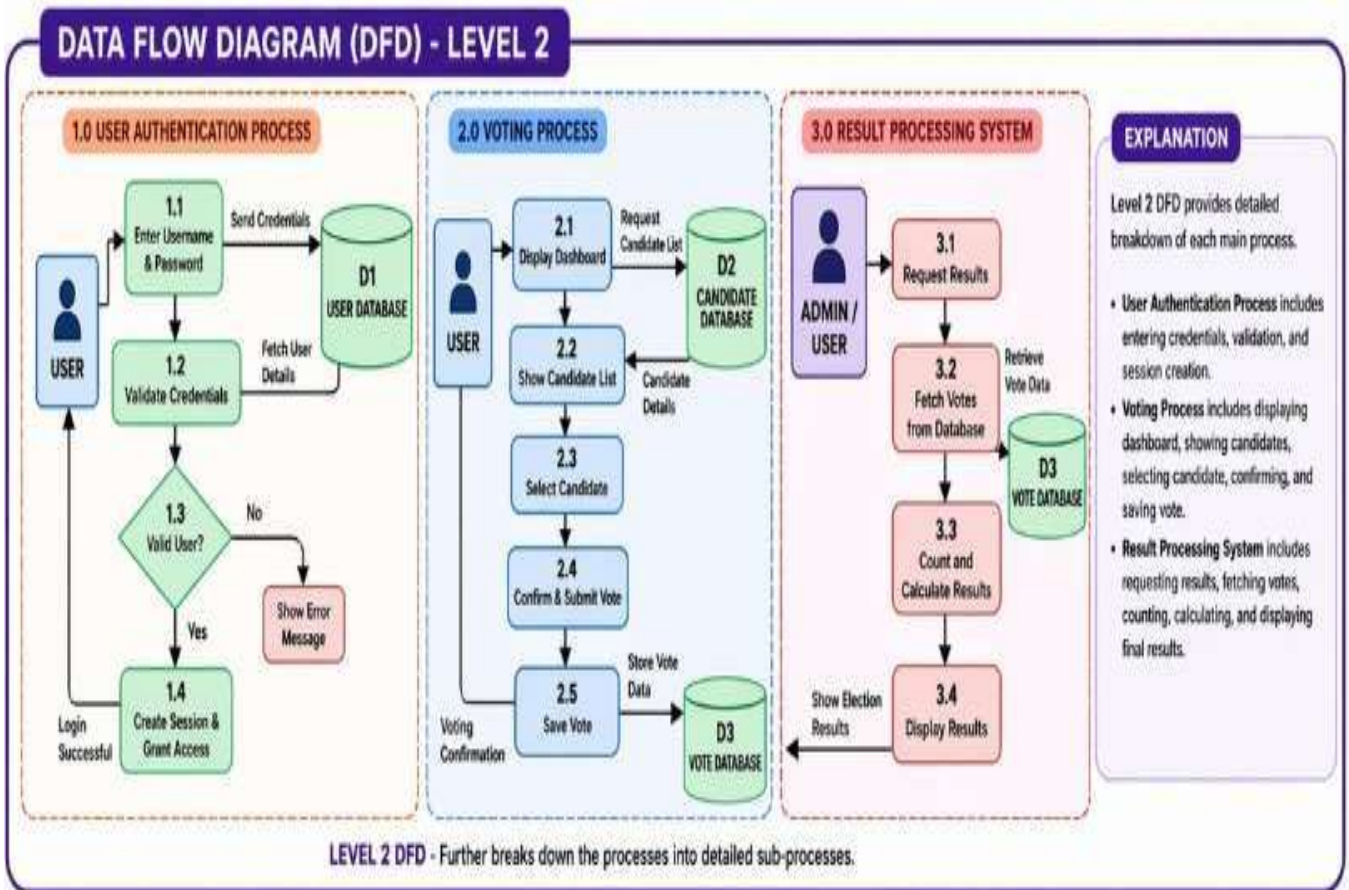
**Main Processes**

- User Authentication Process
- Vote Management Process
- Result Processing System

**Diagram Level 1**

## Level 2 DFD

Level 2 provides detailed breakdown of each process.



## 6.2 Entity Relationship (ER Diagram)

An Entity Relationship (ER) Diagram is a graphical representation of the database structure. It shows how different entities in a system are related to each other. ER diagrams are used during the design phase of a system to model the data and define relationships between different components.

In the **Online Voting System**, the ER Diagram helps in understanding how data such as users, candidates, and votes are stored and connected. It provides a clear view of the database schema and ensures efficient data management.

### Components of ER Diagram

#### 1. Entity

An entity is an object or concept about which data is stored.

##### Examples in this system:

- User
- Admin
- Candidate
- Vote

#### 2. Attribute

Attributes describe the properties of an entity.

##### Examples:

- User → User\_ID, Name, Password
- Candidate → Candidate\_ID, Name, Party

#### 3. Relationship

A relationship shows how entities are connected.

##### Examples:

- User casts Vote
- Vote belongs to Candidate

#### 4. Primary Key

A primary key uniquely identifies each record in a table.

##### Examples:

- User\_ID
- Candidate\_ID

**Main Entities in Online Voting System****1. User Entity****Attributes:**

- User\_ID (Primary Key)
- Name
- Email
- Password
- Voter\_ID

**2. Admin Entity****Attributes:**

- Admin\_ID (Primary Key)
- Username
- Password

**3. Candidate Entity****Attributes:**

- Candidate\_ID (Primary Key)
- Name
- Party\_Name
- Symbol

**4. Vote Entity****Attributes:**

- Vote\_ID (Primary Key)
- User\_ID (Foreign Key)
- Candidate\_ID (Foreign Key)
- Vote\_Date

## Relationships Between Entities

### 1. User → Vote

- Relationship: *User casts Vote*
- Type: One-to-One (one user can vote once)

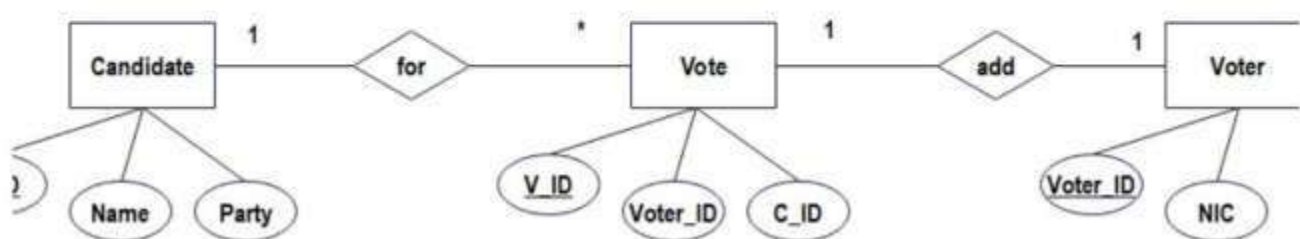
### 2. Candidate → Vote

- Relationship: Candidate receives Vote
- Type: One-to-Many (one candidate can receive many votes)

### 3. Admin → Candidate

- Relationship: Admin manages Candidate
- Type: One-to-Many

# ER Diagram for Online Voting



### 6.3 Data Structure

Data structures are an essential part of any software system as they define how data is organized, stored, and managed efficiently. In the Online Voting System, proper data structures are used to handle user information, candidate details, and voting records.

Efficient data structures help in fast data retrieval, secure storage, and smooth processing of operations such as login authentication, vote casting, and result generation.

#### Objectives of Data Structure

- To organize data efficiently
- To enable fast data access and retrieval
- To ensure data integrity and consistency
- To reduce redundancy
- To improve system performance

#### Types of Data Structures Used

The Online Voting System uses a combination of basic and advanced data structures:

##### 1. Arrays / Lists

Arrays or lists are used to store collections of data elements.

##### Usage in Project:

- Storing list of candidates
- Temporary storage of user data
- Handling multiple records

##### Advantages:

- Easy to access elements
- Simple implementation
- Efficient for small datasets

## Chapter 7: Detailed Design

### 7.1 Input/Output Screen of Project

The Online Voting System provides a user-friendly interface for both voters and administrators. The system is designed using HTML, CSS, and JavaScript for frontend and Flask for backend integration.

#### Input Screens:

##### 1.Registration Page

- New users register before voting.
- Input fields:
  - Username
  - Email ID
  - Aadhar card
  - Mobile number
  - Password
- Data is stored securely in MySQL database.

```
{% extends "base.html" %}
```

```
{% block title%} Register - VoteSecure{% endblock %}
```

```
{% block content %}
```

```
<div class="container-fluid px-0 min-vh-100 bg-gradient"
```

```
style="background: linear-gradient (135deg, #667eea 0%, #764ba2 100%);">
```

```
<div class="row g-0 min-vh-100">
```

```
<div class="col-lg-6 d-none d-lg-flex align-items-center justify-content-center p-5">
```

```
<div class="text-center text-white">
```

```
<i class="fas fa-vote-yea fa-6x mb-4 opacity-75"></i>
```

```
<h1 class="display-4 fw-bold mb-4">VoteSecure</h1>
```

```
<h4 class="lead mb-4">Secure Voting System</h4>
```

```
</div>
```

```
</div>
```

```
<div class="col-lg-6 col-12 d-flex align-items-center justify-content-center p-4 p-lg-5">
```

```
<div class="card shadow-lg border-0 rounded-5 w-100" style="max-width: 450px;">
```

```
<div class="card-body p-5">
```

```
  <div class="text-center mb-5">
```

```
    <i class="fas fa-user-plus fa-4x text-primary mb-3"></i>
```

```
    <h2 class="h1 fw-bold mb-2">Register</h2>
```

```
    <p class="text-muted">Complete registration with OTP verification</p>
```

```
  </div>
```

## 2.OTP Verification

```
{% extends "base.html" %}
```

```
{% block title %}Verify OTP - VoteSecure{% endblock %}
```

```
{% block content %}
```

```
<div class="container-fluid px-0 min-vh-100 bg-gradient"
```

```
  style="background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);">
```

```
<div class="row g-0 min-vh-100">
```

```
  <div class="col-lg-6 d-none d-lg-flex align-items-center justify-content-center p-5">
```

```
    <div class="text-center text-white">
```

```
      <i class="fas fa-mobile-alt fa-6x mb-4 opacity-75"></i>
```

```
      <h1 class="display-4 fw-bold mb-4">Verify OTP</h1>
```

```
      <h4 class="lead mb-4">Check your terminal/console</h4>
```

```
    </div>
```

```
</div>
```

### 3.Login Page

- Users enter Username and Password.
- Authentication is handled using Flask and MySQL database.
- Input fields:
  - Username
  - Password
- Buttons:
  - Login
  - Register (if new user)

```
{% extends "base.html" %}
```

```
{% block title %}Login - Voting System{% endblock %}
```

```
{% block content %}
```

```
<!--  FLASH MESSAGES (Missing tha!) -->
```

```
{% with messages = get_flashed_messages(with_categories=true) %}
```

```
{% if messages %}
```

```
{% for category, message in messages %}
```

```
<div class="alert alert-{{ 'danger' if category == 'danger' else 'success' if category == 'success' else 'warning' }} alert-dismissible fade show mb-4"
```

```
role="alert">
```

```
{{ message|safe }}
```

```
<button type="button" class="btn-close" data-bs-dismiss="alert"></button>
```

```
</div>
```

```
{% endfor %}
```

```
{% endif %}
```

```
{% endwith %}
```

```
<div class="row justify-content-center">
```

```
<div class="col-md-6 col-lg-4">
```

```
<div class="card shadow-lg border-0">
```

```
<div class="card-body p-5">
```

```
<div class="text-center mb-4">
```

```

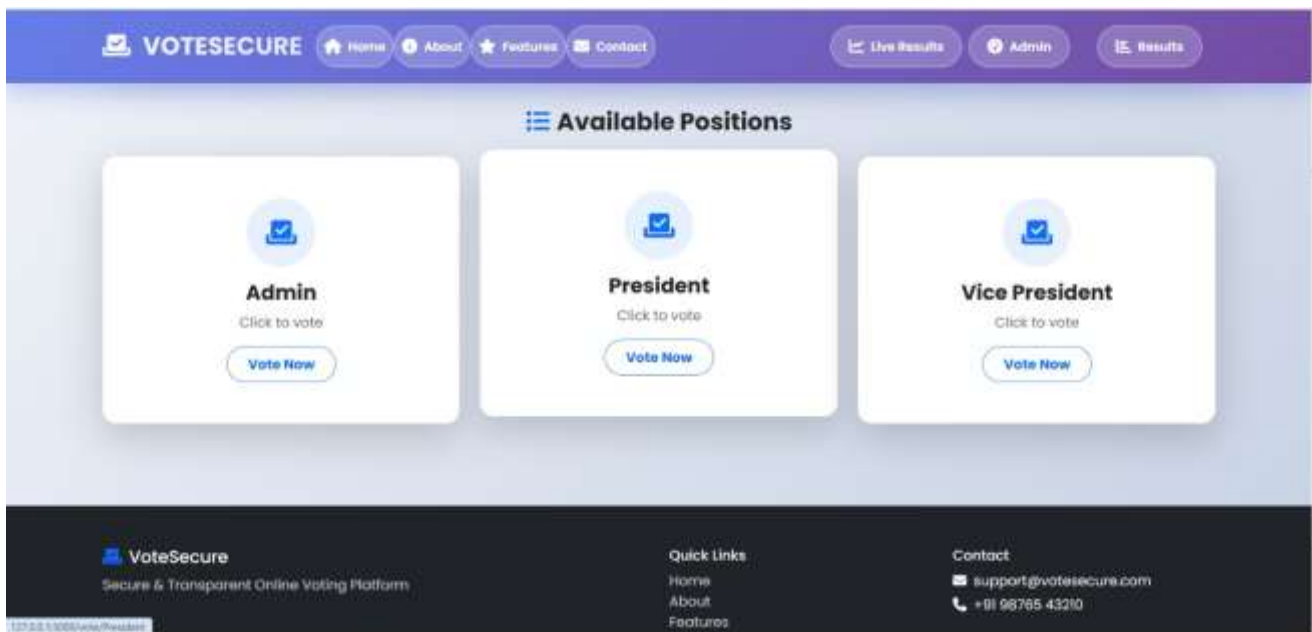
<div class="text-center">
  <i class="fas fa-vote-yea fa-4x text-primary mb-3"></i>
  <h2 class="fw-bold">Welcome Back</h2>
  <p class="text-muted">Login to cast your vote</p>
</div>

```

### Output Screens:

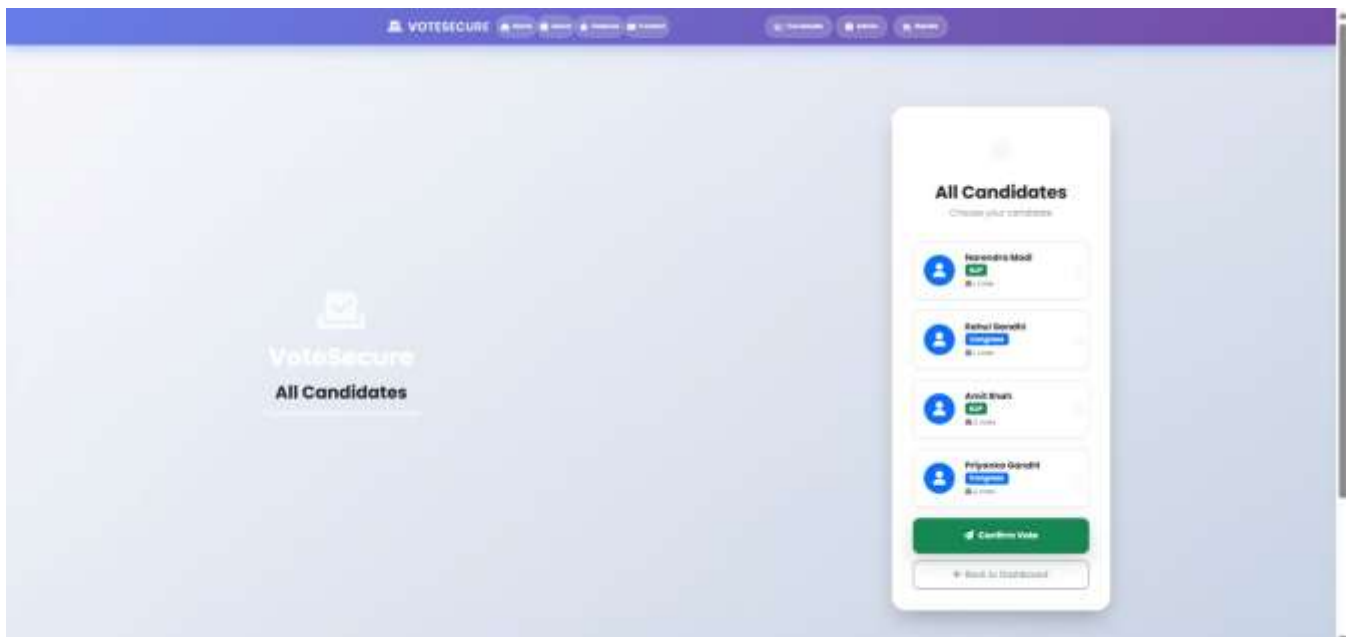
#### 1. Dashboard

- Shows welcome message after login.
- Displays voting status (voted / not voted).



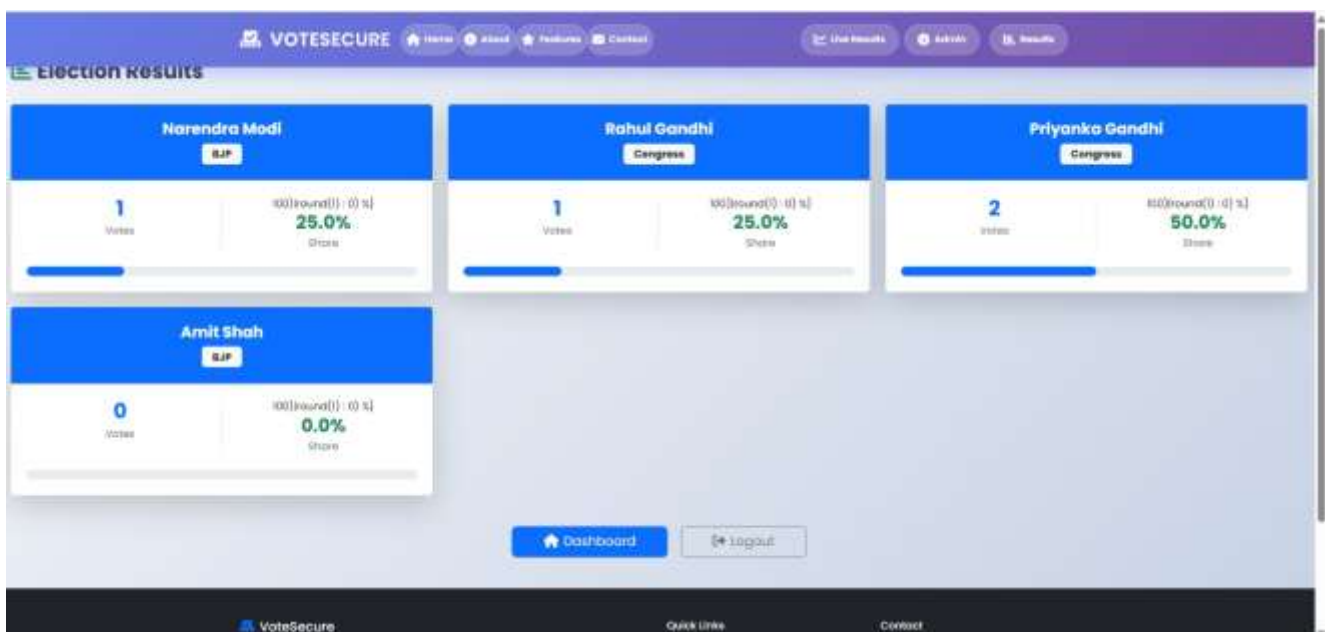
#### 2. Voting Confirmation Page

- Shows message: "Vote Submitted Successfully"
- Prevents multiple voting.



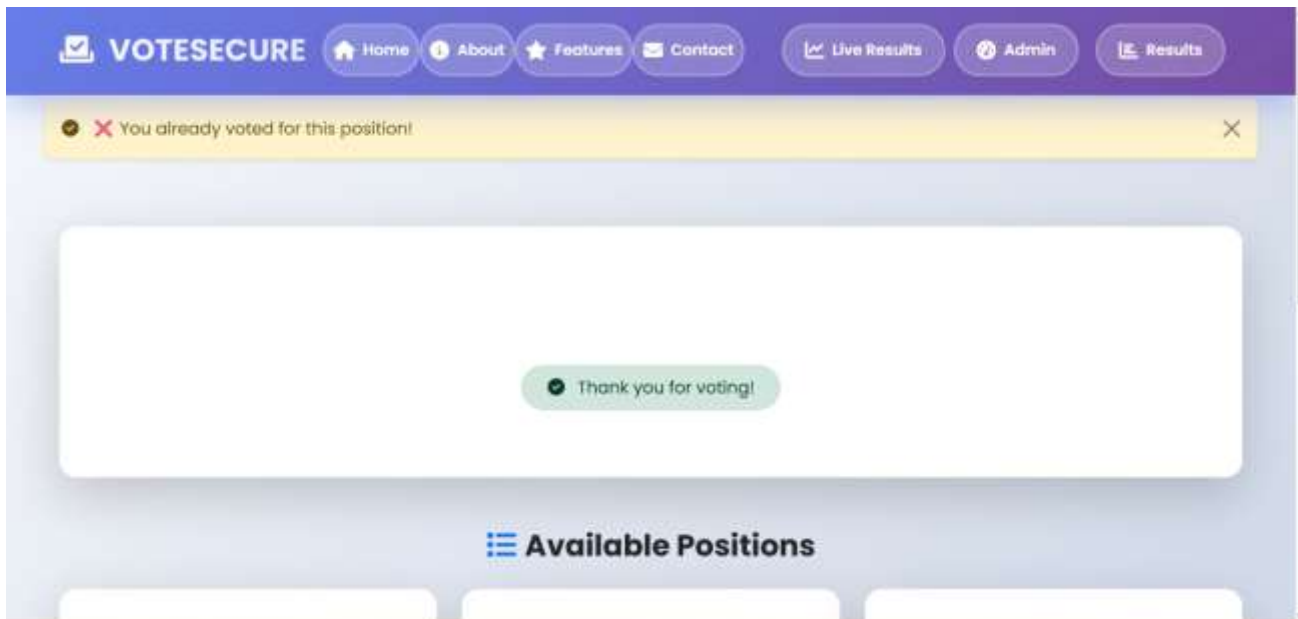
### 3. Result Page

- Displays total votes for each candidate.
- Graph/chart representation (optional).



#### 4. Error Messages

- Invalid login
- Already voted warning
- Required field missing alerts



## 7.2 Validation Checks

Validation ensures data accuracy and system security.

### Frontend Validation (JavaScript):

- Empty field check
- Email format validation
- Password strength validation
- Confirm password match

```
const checkedRadio = document.querySelector('input[name="candidate"]:checked');
```

```
if (!checkedRadio) {
    e.preventDefault();
    alert('Please select a candidate first!');
    return false;
}
```

### Backend Validation (Flask/Python):

- Duplicate user check
- Authentication verification
- Session handling
- Voting restriction (one vote per user)

```
if request.method == 'POST':
```

```
    try:
```

```
        cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

```
        user_id = session['user_id']
```

```
        candidate_id = request.form.get('candidate')
```

```
        # Candidate select nahi kiya
```

```
        if not candidate_id:
```

```
            flash("Please select a candidate!", "warning")
```

```
            return redirect(request.url)
```

```
# Candidate se position nikaalo (IMPORTANT FIX)
```

```
    cursor.execute(
        "SELECT position FROM candidates WHERE id = %s",
        (candidate_id,)
    )
    result = cursor.fetchone()
```

```
if not result:
```

```
    flash("Invalid candidate!", "danger")
    return redirect(url_for('dashboard'))
```

```
position = result['position'] # yahi main fix hai
```

```
# Duplicate vote check (ab sahi kaam karega)
```

```
    cursor.execute(
        "SELECT id FROM votes WHERE user_id = %s AND position = %s",
        (user_id, position)
    )
```

```
if cursor.fetchone():
```

```
    cursor.close()
    flash("You already voted for this position!", "warning")
    return redirect(url_for('dashboard'))
```

```
# Insert vote
```

```
    cursor.execute(
        "INSERT INTO votes (user_id, candidate_id, position) VALUES (%s, %s, %s)",
        (user_id, candidate_id, position)
    )
```

```
cursor.execute(
    "UPDATE candidates SET votes = votes + 1 WHERE id = %s",
    (candidate_id,)
)

# Update user
cursor.execute(
    "UPDATE users SET has_voted = TRUE WHERE id = %s",
    (user_id,)
)

mysql.connection.commit()
cursor.close()

flash(" Vote submitted successfully!", "success")
return redirect(url_for('results'))

except Exception as e:
    mysql.connection.rollback()
    print("Vote Error:", e)
    flash(" Vote failed!", "danger")
    return redirect(request.url)
```

**Database Validation (MySQL):**

- Unique constraints on user ID/email
- Data integrity enforcement

```
ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);
```

```
ALTER TABLE users ADD CONSTRAINT unique_username UNIQUE (username);
```

```
ALTER TABLE users ADD CONSTRAINT unique_aadhaar UNIQUE (aadhaar);
```

```
ALTER TABLE votes ADD CONSTRAINT unique_vote UNIQUE (user_id, position);
```

```
ALTER TABLE users
```

```
ADD CONSTRAINT chk_mobile CHECK (CHAR_LENGTH(mobile) = 10);
```

### 7.3 Program Code

The project is implemented using multiple technologies:

#### Frontend Code (HTML + CSS + JS)

- HTML is used for structure (forms, pages)
- CSS is used for styling (layout, colors)
- JavaScript is used for validation and interactivity

#### Example (Base.html):

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Online Voting System{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.2/css/all.min.css"
rel="stylesheet">
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
rel="stylesheet">
  <style>
    :root {
      --primary: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      --secondary: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
      --success: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
    }
  </style>
```

```
* {  
    font-family: 'Poppins', sans-serif;  
}  
  
body {  
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);  
    min-height: 100vh;  
}  
  
.navbar {  
    background: var(--primary) !important;  
    box-shadow: 0 10px 30px rgba(102, 126, 234, 0.4);  
    padding: 1rem 0;  
}  
  
.navbar-brand {  
    font-weight: 700 !important;  
    font-size: 1.8rem !important;  
    background: linear-gradient(45deg, #fff, #e0e7ff);  
    -webkit-background-clip: text;  
    -webkit-text-fill-color: transparent;  
    background-clip: text;  
}  
  
/* Features Section */  
.features-section {  
    display: flex;  
    gap: 0.5rem;  
    align-items: center;  
    flex-wrap: wrap;  
}
```

```
.feature-card,
.nav-link {
  background: rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(10px);
  color: white !important;
  padding: 0.75rem 1.25rem;
  border-radius: 50px;
  font-weight: 600;
  font-size: 0.9rem;
  cursor: pointer;
  transition: all 0.3s ease;
  border: 2px solid rgba(255, 255, 255, 0.3);
  position: relative;
  overflow: hidden;
  text-decoration: none;
  display: flex;
  align-items: center;
}
.feature-card:hover,
.nav-link:hover {
  background: rgba(255, 255, 255, 0.3);
  transform: translateY(-3px) scale(1.05);
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
  border-color: rgba(255, 255, 255, 0.6);
  color: white !important;
}
.feature-card i,
.nav-link i {
  margin-right: 0.5rem;
  font-size: 1.1rem;
}
```

```
/* Modal Styles */

.feature-modal .modal-content {
  border: none;
  border-radius: 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  overflow: hidden;
}

.feature-modal .modal-header {
  border-bottom: none;
  background: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(10px);
}

.feature-modal .modal-body {
  padding: 2.5rem;
}

.feature-icon {
  font-size: 4rem;
  margin-bottom: 1.5rem;
  opacity: 0.9;
}

/* Sections */

.section {
  padding: 5rem 0;
  min-height: 60vh;
}
```

```
.section-title {
    font-size: 3rem;
    font-weight: 700;
    background: var(--primary);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    margin-bottom: 2rem;
    text-align: center;
}

/* Contact Form */
.contact-form {
    background: rgba(255, 255, 255, 0.9);
    backdrop-filter: blur(10px);
    border-radius: 20px;
    padding: 3rem;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.1);
}

/* Mobile */
@media (max-width: 991px) {
    .features-section {
        margin-top: 1rem;
        justify-content: center;
        gap: 0.75rem;
    }
}

.navbar-nav {
    background: rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    margin-top: 1rem;
    padding: 1rem;
    text-align: center;
}
```

```
.feature-card,
  .nav-link {
    margin: 0.25rem;
    flex: 1;
    min-width: 140px;
    justify-content: center;
  }
}

@keyframes slideInDown {
  from {
    opacity: 0;
    transform: translateY(-30px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.navbar {
  animation: slideInDown 0.6s ease-out;
}
</style>
</head>
```

```
<body>

<nav class="navbar navbar-expand-lg navbar-dark fixed-top">
  <div class="container">
    <a class="navbar-brand fw-bold"
      href="{{ url_for('dashboard') if session.get('user_id') else url_for('index') }}">
      <i class="fas fa-vote-yea me-2"></i>
      <span>VOTE</span><span class="text-warning">SECURE</span>
    </a>
    <button class="navbar-toggler border-0" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="navbarNav">
      <!-- Left Navigation -->
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link" href="#home" data-bs-toggle="modal" data-bs-target="#homeModal">
            <i class="fas fa-home"></i> Home
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#about" data-bs-toggle="modal" data-bs-target="#aboutModal">
            <i class="fas fa-info-circle"></i> About
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#features" data-bs-toggle="modal" data-bs-
target="#featuresModal">
            <i class="fas fa-star"></i> Features
          </a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

```

<li class="nav-item">
    <a class="nav-link" href="#contact" data-bs-toggle="modal" data-bs-
target="#contactModal">
        <i class="fas fa-envelope"></i> Contact
    </a>
</li>
</ul>

<!-- Features Section (Live Results & Admin buttons) -->
<div class="features-section">
    <div class="feature-card" data-bs-toggle="modal" data-bs-target="#resultsModal">
        <i class="fas fa-chart-line"></i> Live Results
    </div>
    <div class="feature-card" data-bs-toggle="modal" data-bs-target="#adminModal">
        <i class="fas fa-tachometer-alt"></i> Admin
    </div>
</div>

<!-- User Section - CLEANED UP -->
<div class="d-flex align-items-center ms-3">
    {% if session.user_id %}
    {% if session.role == 'admin' %}
    <!-- Admin Dropdown Actions -->
    <div class="dropdown me-3">
        <button class="btn btn-outline-light dropdown-toggle px-3 py-1" type="button"
            data-bs-toggle="dropdown">
            <i class="fas fa-plus me-1"></i>Actions
        </button>
        <ul class="dropdown-menu">
            <li><a class="dropdown-item" href="{{ url_for('add_candidate') }}">
                <i class="fas fa-user-plus me-2 text-success"></i>Add Candidate
            </a></li>

```

```

<li><a class="dropdown-item" href="{{ url_for('add_election') }}">
    <i class="fas fa-crown me-2 text-warning"></i>New Election
</a></li>

</ul>

</div>

{% endif %}

<!-- Results Link for all logged-in users -->

<a class="nav-link me-3" href="{{ url_for('results') }}">
    <i class="fas fa-chart-bar"></i> Results
</a>

<!-- Admin Link for admin users only -->
{% if session.role == 'admin' %}
<a class="nav-link me-3" href="{{ url_for('admin') }}">
    <i class="fas fa-user-shield"></i> Admin
</a>
{% endif %}
{% else %}
<!-- Login button for non-logged-in users -->
<a class="btn btn-outline-light btn-lg px-5 py-3 shadow-lg" href="{{ url_for('index') }}">
    <i class="fas fa-sign-in-alt me-2"></i>Login
</a>
{% endif %}

</div>

</div>

</div>

</nav>

```

```
<!-- ALL WORKING MODALS -->
```

```
<!-- Home Modal -->
```

```
<div class="modal fade feature-modal" id="homeModal" tabindex="-1">
```

```
<div class="modal-dialog modal-lg modal-dialog-centered">
```

```
<div class="modal-content">
```

```
<div class="modal-header">
```

```
<i class="fas fa-home feature-icon text-success"></i>
```

```
<h5 class="modal-title">Welcome to VoteSecure</h5>
```

```
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
```

```
</div>
```

```
<div class="modal-body text-center">
```

```
<h3>🏠 The Future of Voting</h3>
```

```
<p class="lead">Secure, transparent, and accessible voting platform for everyone.</p>
```

```
<div class="row mt-4">
```

```
<div class="col-md-4"><i class="fas fa-vote-yea fa-3x text-white-50 mb-3"></i>
```

```
<h6>Vote Now</h6>
```

```
</div>
```

```
<div class="col-md-4"><i class="fas fa-shield-alt fa-3x text-white-50 mb-3"></i>
```

```
<h6>Safe</h6>
```

```
</div>
```

```
<div class="col-md-4"><i class="fas fa-chart-line fa-3x text-white-50 mb-3"></i>
```

```
<h6>Live Results</h6>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- About Modal -->
```

```
<div class="modal fade feature-modal" id="aboutModal" tabindex="-1">
```

```
<div class="modal-dialog modal-xl modal-dialog-centered">
```

```
<div class="modal-content">
```

```
<div class="modal-header">
```

```
<i class="fas fa-info-circle feature-icon text-info"></i>
```

```
<h5 class="modal-title">About VoteSecure</h5>
```

```
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
```

```
</div>
```

```
<div class="modal-body">
```

```
<h3>Our Mission</h3>
```

```
<p class="lead mb-4">Bringing democracy to your fingertips with cutting-edge technology.</p>
```

```
<div class="row">
```

```
<div class="col-md-6">
```

```
<h5>What We Offer:</h5>
```

```
<ul class="list-unstyled">
```

```
<li><i class="fas fa-check-circle text-success me-2"></i>100% Secure Voting</li>
```

```
<li><i class="fas fa-check-circle text-success me-2"></i>Real-time Results</li>
```

```
<li><i class="fas fa-check-circle text-success me-2"></i>Admin Dashboard</li>
```

```
<li><i class="fas fa-check-circle text-success me-2"></i>Smart Verification</li>
```

```
</ul>
```

```
</div>
```

```
<div class="col-md-6">
```

```
<h5>Our Vision:</h5>
```

```
<p>Eliminate election fraud and make voting accessible to every citizen.</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- Features Modal -->
```

```
<div class="modal fade feature-modal" id="featuresModal" tabindex="-1">
```

```
<div class="modal-dialog modal-xl modal-dialog-centered">
```

```
<div class="modal-content">
```

```
<div class="modal-header">
```

```
<i class="fas fa-star feature-icon text-warning"></i>
```

```
<h5 class="modal-title">All Features</h5>
```

```
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
```

```
</div>
```

```
<div class="modal-body">
```

```
<div class="row text-center">
```

```
<div class="col-md-3 mb-4">
```

```
<i class="fas fa-shield-alt fa-3x text-success mb-3"></i>
```

```
<h5>Secure Voting</h5>
```

```
<p>Blockchain + Encryption</p>
```

```
</div>
```

```
<div class="col-md-3 mb-4">
```

```
<i class="fas fa-chart-line fa-3x text-primary mb-3"></i>
```

```
<h5>Live Results</h5>
```

```
<p>Real-time Updates</p> </div>
```

```
<div class="col-md-3 mb-4">
```

```
<i class="fas fa-tachometer-alt fa-3x text-info mb-3"></i>
```

```
<h5>Admin Panel</h5>
```

```
<p>Complete Control</p> </div>
```

```
<div class="col-md-3 mb-4">
```

```
<i class="fas fa-brain fa-3x text-warning mb-3"></i>
```

```
<h5>Smart Verify</h5>
```

```
<p>AI Fraud Detection</p>
```

```
</div>
```

```
</div> </div>
```

```
</div> </div>
```

```
</div>
```

```

<!-- Admin Modal -->

<div class="modal fade feature-modal" id="adminModal" tabindex="-1">
  <div class="modal-dialog modal-lg modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <i class="fas fa-tachometer-alt feature-icon"></i>
        <h5 class="modal-title">Admin Dashboard</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body text-center">
        <h3> ⚡ Complete Control</h3>
        <p class="lead">Manage everything from one dashboard.
          </div>
        </div> </div> </div>
<!-- Main Content -->

<div class="container-fluid mt-5 pt-4">
  {% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
  {% for category, message in messages %}
    <div class="alert alert-{{ 'danger' if category == 'danger' else category }} alert-
dismissible fade show shadow-sm mx-4 mt-3"
      role="alert">
      <i class="fas fa-{{ 'times-circle' if category == 'danger' else 'check-circle' }} me-
2"></i>
      {{ message }}
      <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
  {% endfor %}
  {% endif %}
  {% endwith %}
  {% block content %}{% endblock %}
</div>

```

<!-- Footer -->

```
<footer class="bg-dark text-light py-5 mt-5">
```

```
<div class="container">
```

```
<div class="row">
```

```
<div class="col-md-6">
```

```
<h5><i class="fas fa-vote-yea me-2 text-primary"></i>VoteSecure</h5>
```

```
<p class="mb-0 opacity-75">Secure & Transparent Online Voting
```

Platform</p>

```
</div>
```

```
<div class="col-md-3">
```

```
<h6>Quick Links</h6>
```

```
<ul class="list-unstyled">
```

toggle="modal"

```
<li><a href="#" class="text-light text-decoration-none opacity-75" data-bs-
```

```
data-bs-target="#homeModal">Home</a></li>
```

toggle="modal"

```
<li><a href="#" class="text-light text-decoration-none opacity-75" data-bs-
```

```
data-bs-target="#aboutModal">About</a></li>
```

toggle="modal"

```
<li><a href="#" class="text-light text-decoration-none opacity-75" data-bs-
```

```
data-bs-target="#featuresModal">Features</a></li>
```

```
</ul>
```

```
</div>
```

```
<div class="col-md-3">
```

```
<h6>Contact</h6>
```

```
<p class="mb-1"><i class="fas fa-envelope me-2"></i>support@votesecure.com</p>
```

```
<p><i class="fas fa-phone me-2"></i>+91 98765 43210</p>
```

```
</div>
```

```
</div>
```

```
<hr class="my-4">
```

```
<div class="text-center">
```

```
<p class="mb-0 opacity-75">&copy; 2024 VoteSecure. All rights reserved.</p>
```

```

        </div>
    </div>
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
<script>
    // Ripple effect for all clickable cards
    document.addEventListener('click', function (e) {
        let target = e.target.closest('.feature-card, .nav-link');
        if (target) {
            // Create ripple
            const ripple = document.createElement('span');
            ripple.style.position = 'absolute';
            ripple.style.borderRadius = '50%';
            ripple.style.background = 'rgba(255,255,255,0.6)';
            ripple.style.transform = 'scale(0)';
            ripple.style.animation = 'ripple 0.6s linear';
            ripple.style.left = (e.clientX - target.getBoundingClientRect().left) + 'px';
            ripple.style.top = (e.clientY - target.getBoundingClientRect().top) + 'px';
            ripple.style.width = ripple.style.height = '50px';

            target.style.position = 'relative';
            target.style.overflow = 'hidden';
            target.appendChild(ripple);

            setTimeout(() => {
                ripple.remove();
            }, 600);
        }
    });

```

```
// Add ripple animation CSS

const style = document.createElement('style');
style.textContent = `
    @keyframes ripple {
        to {
            transform: scale(4);
            opacity: 0;
        }
    }
`;
document.head.appendChild(style);

// Auto-close modals on outside click (optional)
var modals = document.querySelectorAll('.modal');
modals.forEach(function (modal) {
    modal.addEventListener('hidden.bs.modal', function () {
        console.log('Modal closed: ' + this.id);
    });
});

// Smooth scrolling backup (if needed)
document.querySelectorAll('a[href^="#"]').forEach(anchor => {
    anchor.addEventListener('click', function (e) {
        e.preventDefault();
        const targetId = this.getAttribute('href').substring(1);
        const targetModal = document.getElementById(targetId + 'Modal');
        if (targetModal) {
            var modal = new bootstrap.Modal(targetModal);
            modal.show();
        }
    });
});
```

//  FIXED: सिर्फ CONTACT FORM को target करें - LOGIN FORMS SAFE!

```
const contactForm = document.querySelector('#contact-form');
if (contactForm) {
    contactForm.addEventListener('submit', function (e) {
        e.preventDefault();
        alert('Message sent successfully! We will contact you soon.');
```

this.reset();

```
var contactModal =
bootstrap.Modal.getInstance(document.getElementById('contactModal'));
    if (contactModal) {
        contactModal.hide();
    }
});
}
```

// Navbar scroll fix for fixed-top

```
window.addEventListener('scroll', function () {
    const navbar = document.querySelector('.navbar');
    if (window.scrollY > 50) {
        navbar.style.background = 'rgba(102, 126, 234, 0.95)';
    } else {
        navbar.style.background = 'var(--primary)';
    }
});
</script>
</body>

</html>
```

**Backend Code (Python + Flask)**

- Flask handles routing and server logic
- Connects frontend with database

**Example (Login Route):**

```

from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify
from flask_mysql import MySQL
from werkzeug.security import generate_password_hash, check_password_hash
from config import Config
import MySQLdb.cursors
import re
import random
import smtplib
from email.mime.text import MIMEText
from datetime import datetime, timedelta
import os
import time

app = Flask(__name__)
app.secret_key = os.urandom(24).hex() # secure random key
app.config.from_object(Config)

mysql = MySQL(app)

# EMAIL CONFIG - UPDATE THESE
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
EMAIL_USER = os.getenv('EMAIL_USER', 'puttankumar76@gmail.com')
EMAIL_PASSWORD = os.getenv('EMAIL_PASSWORD', 'your_16_digit_app_password_here')

def generate_otp():
    """Generate 6-digit OTP"""
    otp = str(random.randint(100000, 999999))
    print(f"TERMINAL OTP: {otp}")
    print("=" * 50)
    return otp

```

```
def send_otp_email(to_email, otp):  
    """Send OTP via email"""  
  
    try:  
        msg = MIMEText(f"""  
VoteSecure OTP: *{otp}*  
  
Valid for 10 minutes.  
Don't share your OTP with anyone!  
  
Team VoteSecure  
""")  
        msg['Subject'] = 'VoteSecure - Your OTP Code'  
        msg['From'] = EMAIL_USER  
        msg['To'] = to_email  
  
        server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)  
        server.starttls()  
        server.login(EMAIL_USER, EMAIL_PASSWORD)  
        server.send_message(msg)  
        server.quit()  
        print(f"EMAIL SENT TO: {to_email}")  
        return True  
    except Exception as e:  
        print(f"EMAIL FAILED: {e} - Using Terminal OTP")  
        return False
```

```

# SEND OTP ROUTE - Frontend register.html से call
@app.route('/send-otp', methods=['POST'])
def send_otp():
    """Send OTP via email - Supports JSON and FormData"""
    try:
        # Handle multiple content types
        if request.content_type == 'application/json':
            data = request.get_json(silent=True) or {}
        else:
            data = request.form.to_dict()

        # 🔥 Get email from any source
        email = (data.get('email') or
                 request.args.get('email') or
                 request.form.get('email') or
                 "").strip().lower()

        print(f"SEND-OTP REQUEST:")
        print(f"  Email: {email}")
        print(f"  Content-Type: {request.content_type}")
        print(f"  Data: {data}")

        if not email:
            return jsonify({'success': False, 'error': 'Email parameter missing!'}), 400
        if not re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email):
            return jsonify({'success': False, 'error': 'Invalid email format!'}), 400

```

# Check if user exists

```
cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
cursor.execute("SELECT id, username FROM users WHERE email = %s", (email,))
existing_user = cursor.fetchone()
cursor.close()
```

if existing\_user:

```
    return jsonify({
        'success': False,
        'error': f'Email already registered as "{existing_user["username"]}"!',
        'login_url': url_for('login')
    }), 400
```

# Generate OTP

```
otp = generate_otp()
email_sent = send_otp_email(email, otp)
```

# Store in session

```
session['otp_email'] = email
session['otp'] = otp
session['otp_time'] = time.time() + 600 # 10 minutes
session['otp_attempts'] = 0

print(f"SESSION STORED - Email: {email}, OTP: {otp}, Expires:
{datetime.fromtimestamp(session['otp_time'])}")

return jsonify({
    'success': True,
    'message': 'OTP sent successfully! Check terminal.',
    'email': email,
    'email_sent': email_sent,
    'terminal_otp': otp,
    'expires_in': 600
}), 200
```

except Exception as e:

```
print(f"SEND-OTP ERROR: {e}")
import traceback
traceback.print_exc()
return jsonify({
    'success': False,
    'error': 'Internal server error. Check terminal for OTP.'
}), 500
```

# VERIFY OTP ROUTE -

```
@app.route('/verify-otp', methods=['POST'])
```

```
def verify_otp():
```

```
    """Frontend verifyOtp() से call - OTP verification"""
```

```
    try:
```

```
        data = request.get_json()
```

```
        user_otp = data.get('otp', '').strip()
```

```
        print(f"VERIFY-OTP CALLED:")
```

```
        print(f"  User OTP: {user_otp}")
```

```
        print(f"  Session OTP: {session.get('otp', 'None')}")
```

```
    # Get session data
```

```
    email = session.get('otp_email')
```

```
    stored_otp = session.get('otp')
```

```
    otp_time = session.get('otp_time', 0)
```

```
    attempts = session.get('otp_attempts', 0)
```

```
    if not email or not stored_otp:
```

```
        print("No OTP session found!")
```

```
        return jsonify({'success': False, 'error': 'Session expired! Resend OTP.'})
```

```
# Check attempts limit (3 tries)
```

```
if attempts >= 3:
```

```
    session.clear()
```

```
    print("Too many attempts!")
```

```
    return jsonify({'success': False, 'error': 'Too many failed attempts! Resend OTP.'})
```

```
# Check OTP expiry (10 minutes)
```

```
if time.time() > otp_time:
```

```
    session.clear()
```

```
    print("OTP expired!")
```

```
    return jsonify({'success': False, 'error': 'OTP expired! Please request new one.'})
```

```
# OTP MATCH CHECK
```

```
if user_otp == stored_otp:
```

```
    print(f"OTP VERIFIED SUCCESSFULLY for {email}")
```

```
    session['otp_verified'] = True # Mark as verified
```

```
    session['otp_attempts'] = 0 # Reset attempts
```

```
    return jsonify({
```

```
        'success': True,
```

```
        'message': 'OTP verified! Proceeding to registration...'
```

```
    })
```

```
else:
```

```
    # Wrong OTP
```

```
    session['otp_attempts'] = attempts + 1
```

```
    remaining = 3 - (attempts + 1)
```

```
    print(f"WRONG OTP! Remaining attempts: {remaining}")
```

```
    return jsonify({
```

```
        'success': False,
```

```
        'error': f'Invalid OTP! {remaining} attempts remaining.'
```

```
    })
```

except Exception as e:

```
print(f"VERIFY-OTP ERROR: {e}")
```

```
import traceback
```

```
traceback.print_exc()
```

```
return jsonify({'success': False, 'error': 'Verification failed!'}), 500
```

# REGISTER ROUTE (Updated for AJAX flow)

```
@app.route('/register', methods=['GET', 'POST'])
```

```
def register():
```

```
    if request.method == 'GET':
```

```
        email = session.get('otp_email', "")
```

```
        return render_template('register.html', email=email)
```

```
    if request.method == 'POST':
```

```
        try:
```

```
            # Get form data (from hidden form submission)
```

```
            username = request.form.get('username', "").strip()
```

```
            email = request.form.get('email') or session.get('otp_email')
```

```
            password = request.form.get('password', "")
```

```
            aadhaar = request.form.get('aadhaar', "")
```

```
            mobile = request.form.get('mobile', "")
```

```
            print(f"REGISTER ATTEMPT:")
```

```
            print(f" Username: {username}")
```

```
            print(f" Email: {email}")
```

```
            print(f" OTP Verified: {session.get('otp_verified', False)}")
```

```
            # MANDATORY: Check OTP verification
```

```
            if not session.get('otp_verified'):
```

```
                print("OTP not verified!")
```

```
                return "
```

```
<script>
    alert("Please verify OTP first!");
    window.history.back();
</script>
'''

# Client-side validation (server-side too)
if not all([username, email, password, aadhaar, mobile]):
    return 'All fields required!', 400

if len(password) < 6:
    return 'Password must be 6+ characters!', 400

if len(aadhaar) != 12 or not aadhaar.isdigit():
    return 'Aadhaar must be 12 digits!', 400

if len(mobile) != 10 or not mobile.isdigit():
    return 'Mobile must be 10 digits!', 400

# Check duplicates
cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)

# Username check
cursor.execute("SELECT id FROM users WHERE username = %s", (username,))
if cursor.fetchone():
    cursor.close()
    return 'Username already taken!', 400
```

# Check duplicates

```
cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

# Username check

```
cursor.execute("SELECT id FROM users WHERE username = %s", (username,))
```

```
if cursor.fetchone():
```

```
    cursor.close()
```

```
    return 'Username already taken!', 400
```

# Email check (already done in send-otp, but double-check)

```
cursor.execute("SELECT id FROM users WHERE email = %s", (email,))
```

```
if cursor.fetchone():
```

```
    cursor.close()
```

```
    return 'Email already registered!', 400
```

# Insert new user

```
cursor.execute("""
```

```
    INSERT INTO users (username, email, password, aadhaar, mobile, role, has_voted, created_at)
```

```
    VALUES (%s, %s, %s, %s, %s, 'voter', FALSE, NOW())
```

```
""", (username, email, generate_password_hash(password), aadhaar, mobile))
```

```
mysql.connection.commit()
```

```
new_user_id = mysql.connection.insert_id()
```

```
cursor.close()
```

```
print(f"NEW USER REGISTERED: ID={new_user_id}, {username}")
```

# Clear session

```
session.clear()
```

```
return f'''
```

```
<script>
```

```
    alert("Registration Successful!\nWelcome {username}!");
```

```
    window.location.href = "/login";
```

```
</script>
```

```
'''
```

```
except Exception as e:
```

```
    if mysql.connection:
```

```
        mysql.connection.rollback()
```

```
    print(f"REGISTER ERROR: {e}")
```

```
    import traceback
```

```
    traceback.print_exc()
```

```
    return f'Registration failed: {str(e)}', 500
```

```
# INDEX ROUTE
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def index():
```

```
    if request.method == 'POST':
```

```
        email = request.form.get('email', "").strip().lower()
```

```
        print(f"INDEX POST OTP: {email}")
```

```
    if not email or not re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email):
```

```
        flash('Valid email required!', 'danger')
```

```
        return render_template('index.html')
```

```
# Check duplicate
```

```
try:
```

```
    cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

```
    cursor.execute("SELECT id FROM users WHERE email = %s", (email,))
```

```
    if cursor.fetchone():
```

```
        cursor.close()
```

```
        flash('Email already registered! <a href="/login">Login here</a>', 'warning')
```

```
        return redirect(url_for('login'))
```

```
    cursor.close()
```

```
except:
```

```
    pass
```

```
# Generate & store OTP
```

```
    otp = generate_otp()
```

```
    send_otp_email(email, otp)
```

```
    session['otp_email'] = email
```

```
    session['otp'] = otp
```

```
    session['otp_time'] = time.time() + 600
```

```
    flash(f'OTP sent! Terminal: <strong>{otp}</strong><br>Click Register below!', 'success')
```

```
    return redirect(url_for('register'))
```

```
return render_template('index.html')
```

```
# LOGIN ROUTE
```

```
@app.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    if request.method == 'POST':
```

```
        email = request.form.get('email', '').strip().lower()
```

```
        password = request.form.get('password', '')
```

```
        print(f" LOGIN: {email}")
```

```
    if not email or not password:
```

```
        flash('Email & password required!', 'danger')
```

```
        return render_template('login.html')
```

```
    try:
```

```
        cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

```
        cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

```
        user = cursor.fetchone()
```

```
        cursor.close()
```

```
        if user and check_password_hash(user['password'], password):
```

```
session['user_id'] = user['id']

    session['username'] = user['username']
    session['role'] = user['role']
    print(f"LOGIN SUCCESS: {user['username']}")
    flash('Login successful!', 'success')
    return redirect(url_for('dashboard'))
else:
    flash('Invalid credentials!', 'danger')
    print("LOGIN FAILED")
except Exception as e:
    print(f" LOGIN ERROR: {e}")
    flash('Login error! Try again.', 'danger')

return render_template('login.html')

# RESULTS ROUTE
@app.route('/results')
def results():
    if 'user_id' not in session:
        flash('Login required!', 'warning')
        return redirect(url_for('login'))

    try:
        cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
# 🔥 FIX: direct candidates table se votes lo
        cursor.execute("""
            SELECT * FROM candidates
            ORDER BY position, votes DESC
        """)
        results = cursor.fetchall()
```

# Stats

```
cursor.execute("SELECT COUNT(*) as total_candidates FROM candidates")
```

```
total_candidates = cursor.fetchone()['total_candidates']
```

```
cursor.execute("SELECT COUNT(*) as total_votes FROM votes")
```

```
total_votes = cursor.fetchone()['total_votes'] or 0
```

```
cursor.close()
```

```
return render_template(
```

```
    'results.html',
```

```
    results=results,
```

```
    stats={
```

```
        'total_candidates': total_candidates,
```

```
        'total_votes': total_votes
```

```
    }
```

```
)
```

```
except Exception as e:
```

```
    print("Results Error:", e)
```

```
    flash('Error loading results!', 'danger')
```

```
    return render_template('results.html', results=[], stats={})
```

# otp verify

```
@app.route('/verify_email_otp', methods=['GET', 'POST'])
```

```
def verify_email_otp():
```

```
    if request.method == 'POST':
```

```
        user_otp = request.form.get('otp', "").strip()
```

```
        stored_otp = session.get('otp')
```

```
        otp_time = session.get('otp_time', 0)
```

```
print(f"OTP VERIFY: {user_otp} vs {stored_otp}")

if not user_otp:
    flash('Enter OTP!', 'danger')
    return render_template('verify_otp.html')

if time.time() > otp_time:
    flash('OTP expired! Request new one.', 'danger')
    session.clear()
    return redirect(url_for('index'))

if user_otp == stored_otp:
    flash(' OTP Verified! Continue to register.', 'success')
    return redirect(url_for('register'))
else:
    flash(' Wrong OTP! Try again.', 'danger')

return render_template('verify_otp.html')

@app.route('/logout')
def logout():
    session.clear()
    flash('Logged out!', 'success')
    return redirect(url_for('login'))
```

## # 🔥 DASHBOARD ROUTES

@app.route('/dashboard')

def dashboard():

if 'user\_id' not in session:

return redirect(url\_for('login'))

try:

cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)

# User info

cursor.execute("SELECT has\_voted, username FROM users WHERE id = %s", (session['user\_id'],))

user = cursor.fetchone()

# 🔥 Fetch positions from positions table (FIXED)

cursor.execute("SELECT name FROM positions ORDER BY name")

positions = cursor.fetchall()

cursor.close()

return render\_template('dashboard.html', user=user or {}, positions=positions or [])

except Exception as e:

print(f"Dashboard Error: {e}")

return render\_template('dashboard.html', user={}, positions=[])

# candidate

@app.route('/candidates/&lt;position&gt;')

def get\_candidates(position):

try:

cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)

cursor.execute("SELECT \* FROM candidates WHERE position = %s", (position,))

data = cursor.fetchall()

cursor.close()

return jsonify(data)

except Exception as e:

print(f"Candidate Fetch Error: {e}")

return jsonify([])

## # VOTE ROUTE

```
@app.route('/vote', defaults={'position': None}, methods=['GET', 'POST'])
```

```
@app.route('/vote/<position>', methods=['GET', 'POST'])
```

```
def vote(position):
```

```
    if 'user_id' not in session:
```

```
        return redirect(url_for('login'))
```

```
    if request.method == 'POST':
```

```
        try:
```

```
            cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

```
            user_id = session['user_id']
```

```
            candidate_id = request.form.get('candidate')
```

```
            # Candidate select nahi kiya
```

```
            if not candidate_id:
```

```
                flash("Please select a candidate!", "warning")
```

```
                return redirect(request.url)
```

```
            # Candidate se position nikaalo (IMPORTANT FIX)
```

```
            cursor.execute(
```

```
                "SELECT position FROM candidates WHERE id = %s",
```

```
                (candidate_id,)
```

```
            )
```

```
            result = cursor.fetchone()
```

```
            if not result:
```

```
                flash("Invalid candidate!", "danger")
```

```
                return redirect(url_for('dashboard'))
```

```
            position = result['position'] # yahi main fix hai
```

```
            # Duplicate vote check (ab sahi kaam karega)
```

```
            cursor.execute(
```

```
                "SELECT id FROM votes WHERE user_id = %s AND position = %s",
```

```
                (user_id, position)
```

```
            )
```

```
if cursor.fetchone():
    cursor.close()
    flash("You already voted for this position!", "warning")
    return redirect(url_for('dashboard'))

# Insert vote
cursor.execute(
    "INSERT INTO votes (user_id, candidate_id, position) VALUES (%s, %s, %s)",
    (user_id, candidate_id, position)
)
cursor.execute(
    "UPDATE candidates SET votes = votes + 1 WHERE id = %s",
    (candidate_id,)
)

# Update user
cursor.execute(
    "UPDATE users SET has_voted = TRUE WHERE id = %s",
    (user_id,)
)

mysql.connection.commit()
cursor.close()

flash(" Vote submitted successfully!", "success")
return redirect(url_for('results'))

except Exception as e:
    mysql.connection.rollback()
    print("Vote Error:", e)
    flash(" Vote failed!", "danger")
    return redirect(request.url)
```

```
# GET PART
```

```
try:
```

```
    cursor = mysql.connection.cursor(MySQLdb.cursors.DictCursor)
```

```
    if position:
```

```
        cursor.execute("SELECT * FROM candidates WHERE position=%s", (position,))
```

```
    else:
```

```
        cursor.execute("SELECT * FROM candidates")
```

```
    candidates = cursor.fetchall()
```

```
    cursor.close()
```

```
except:
```

```
    candidates = []
```

```
return render_template('vote.html', candidates=candidates, position=position)
```

```
if __name__ == '__main__':
```

```
    # print("VoteSecure - ALL ROUTES LOADED!")
```

```
    # print("FIXED: /send-otp → /verify-otp → /register")
```

```
    # print("Flow: Register → Send OTP → Terminal OTP → Verify → Register → Login")
```

```
    # print("Check TERMINAL for OTP!")
```

```
    # print("Update EMAIL_PASSWORD with Gmail")
```

```
    # print("📧 Update EMAIL_PASSWORD with Gmail App Password")
```

```
    # print("=" * 80)
```

```
    app.run(debug=True, host='0.0.0.0', port=5000)
```

**Database (MySQL)**

- Stores user data, candidates, and votes

**Tables:****1. Users Table**

- id
- username
- email
- password
- voted (Yes/No)

```
CREATE DATABASE voting_system;
```

```
USE voting_system;
```

```
-- Users Table
```

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role ENUM('voter', 'admin') DEFAULT 'voter',  
    has_voted BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

**Candidates Table**

- candidate\_id
- candidate\_name
- party\_name

-- Candidates Table

```
CREATE TABLE candidates (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  party VARCHAR(50),
  photo VARCHAR(255),
  position VARCHAR(50),
  votes INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

**Votes Table**

- vote\_id
- user\_id
- candidate\_id

-- Votes Table

```
CREATE TABLE votes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  candidate_id INT,
  position VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users (id),
  FOREIGN KEY (candidate_id) REFERENCES candidates (id),
  UNIQUE KEY unique_vote (user_id, position)
);
```

- Insert Sample Data

INSERT INTO

```
users (  
    username,  
    email,  
    password,  
    role  
)
```

VALUES (

```
'admin',  
'admin@voting.com',  
'$2b$12$92lXUNpkj00rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',  
'admin'  
,  
(  
    'voter1',  
    'voter1@gmail.com',  
    '$2b$12$92lXUNpkj00rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',  
    'voter'  
);
```

INSERT INTO

candidates (name, party, position)

VALUES (

```
'Amit Shah',  
'BJP',  
'President'  
,
```

```

(
    'Rahul Gandhi',
    'Congress',
    'President'
),
(
    'Arvind Kejriwal',
    'AAP',
    'President'
),
(
    'Mamata Banerjee',
    'TMC',
    'President'
);

INSERT INTO candidates (name, position, party) VALUES
('Narendra Modi', 'President', 'BJP'),
('Rahul Gandhi', 'President', 'Congress'),
('Amit Shah', 'Vice President', 'BJP'),
('Priyanka Gandhi', 'Vice President', 'Congress');

SELECT * FROM candidates;

TRUNCATE TABLE votes;

ALTER TABLE votes ADD UNIQUE (user_id, position);

SELECT * FROM votes;

SELECT * FROM candidates;

SELECT name, votes FROM candidates;

```

## Chapter 8: Testing Techniques

### 8. Introduction to Testing

Testing is an important phase in software development. It ensures that the system works correctly and meets user requirements. In the Online Voting System, testing is performed to verify that all modules such as login, registration, voting, and result processing function accurately and securely.

The main objective of testing is to identify errors, bugs, and security issues before the system is deployed for real users. Proper testing improves system reliability, performance, and user satisfaction.

#### 8.1 Objectives of Testing

The key objectives of testing the Online Voting System are:

- To ensure that all functionalities work as expected
- To detect and fix errors and bugs
- To verify system security and data integrity
- To check performance under different conditions
- To ensure only authorized users can vote
- To prevent duplicate voting
- To validate user inputs and outputs

#### 8.2 Types of Testing Used

Different types of testing were performed in this project:

##### 1. Unit Testing

Unit testing is done to test individual components of the system separately.

- Testing login function
- Testing registration form
- Testing vote submission logic

##### 2. Integration Testing

Integration testing checks how different modules work together.

- Frontend (HTML, CSS, JS) integrated with Flask backend
- Flask backend connected with MySQL database

##### 3. System Testing

System testing evaluates the complete system as a whole.

- Complete workflow: Registration → Login → Voting → Result
- Ensures all modules are properly connected and working

#### 4. Functional Testing

Functional testing verifies whether the system performs according to requirements.

- User can register successfully
- User can login with correct credentials
- User can vote only once
- Admin can view results

#### 5. Validation Testing

Validation testing ensures correct input is accepted.

- Email format validation
- Password confirmation
- Required field checks

#### 6. Security Testing

Security testing is very important in a voting system.

- Prevent unauthorized access
- Ensure secure login system
- Protect database from invalid queries
- Avoid multiple voting by same user

#### 7. Performance Testing

Performance testing checks system behavior under load.

- Multiple users accessing system simultaneously
- Response time of login and voting
- Database query performance

#### 8. User Acceptance Testing (UAT)

UAT is done by real users to verify the system.

- Users tested system interface
- Feedback collected for improvements
- Ensures system is user-friendly

### 8.3 Testing Tools Used

The following tools and technologies were used for testing:

- **Manual Testing** – Checking system manually
- **Browser Testing** – Chrome, Edge compatibility
- **Flask Debug Mode** – For backend error detection
- **MySQL Workbench** – Database testing and queries

### 8.4 Bug Tracking and Fixing

During testing, some bugs were identified and fixed:

- Login issues due to incorrect query
- Duplicate voting bug fixed using user status
- Form validation errors corrected
- Database connection issues resolved

### 8.5 Result of Testing

After performing all testing techniques:

- The system works correctly
- All modules are functioning properly
- No major bugs found
- System is secure and reliable
- User interface is easy to use

## Chapter 9: Implementation & Maintenance

### 9. Introduction

Implementation is the process of converting the designed system into a working system. It involves installing the software, configuring the environment, and making the system available for users.

In the Online Voting System, implementation ensures that all modules such as registration, login, voting, and result management are properly deployed and functional.

#### 9.1 System Implementation

The Online Voting System is implemented using modern web technologies including HTML, CSS, JavaScript for frontend, and Python with Flask for backend. MySQL is used as the database.

##### Steps of Implementation:

##### 1. Environment Setup

- Install Python and Flask framework
- Install MySQL database
- Install required libraries (Flask, MySQL Connector)
- Setup code editor (VS Code)

##### 2. Database Setup

- Create database named “online\_voting\_system”
- Create required tables:
  - Users
  - Candidates
  - Votes

##### 3. Frontend Development

- Design user interfaces using HTML and CSS
- Add interactivity using JavaScript
- Create pages:
  - Login Page
  - Registration Page
  - Voting Page
  - Result Page

**4. Backend Development**

- Develop Flask routes for different functionalities
- Connect frontend with database
- Handle user authentication and session

**5. Integration**

- Connect frontend forms with backend logic
- Ensure data is correctly stored and retrieved from database

**6. Testing Before Deployment**

- Perform all testing techniques
- Fix bugs and errors

**7. Deployment**

- Run application on local server (localhost)
- Can be deployed on cloud/server for real use

**9.2 System Requirements****Hardware Requirements:**

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: 500 GB
- Internet Connection

**Software Requirements:**

- Operating System: Windows/Linux
- Frontend: HTML, CSS, JavaScript
- Backend: Python (Flask)
- Database: MySQL
- Browser: Chrome, Edge

### 9.3 Installation Procedure

The system can be installed using the following steps:

1. Install Python from official website
2. Install Flask using command:

```
pip install flask
```

3. Install MySQL and create database
4. Import database tables
5. Place project files in a folder
6. Run Flask application:

```
python app.py
```

7. Open browser and go to:

<http://127.0.0.1:5000/>

### 9.4 User Training

User training is important to ensure proper usage of the system.

- Users are guided on how to register and login
- Instructions are provided for casting votes
- Admin is trained to manage candidates and view results
- Simple and user-friendly interface reduces training effort

### 9.5 Maintenance of the System

Maintenance ensures the system runs smoothly after implementation.

#### Types of Maintenance:

#### 1. Corrective Maintenance

- Fixing bugs and errors
- Example: login issue, database errors

#### 2. Adaptive Maintenance

- Updating system according to new requirements
- Example: adding new features

#### 3. Perfective Maintenance

- Improving performance and user interface
- Example: better design, faster loading

### 9.6 Security and Backup

Security is critical in a voting system.

- User authentication system
- Password protection
- One user one vote rule
- Database security
- Regular data backup

### 9.7 Limitations of Implementation

- Requires internet connection
- Basic security (can be improved)
- Limited scalability for large elections
- Dependent on server performance

### 9.8 Future Maintenance Scope

- Implement advanced security (encryption, OTP)
- Add biometric or Aadhaar verification
- Cloud-based deployment
- Mobile application version
- Real-time analytics dashboard

## Chapter 10: Limitations, Future Scope & Enhancements

### 10. Introduction

Every system has certain limitations due to technical, financial, or design constraints. The Online Voting System developed in this project is efficient and user-friendly, but it still has some limitations. However, there is a wide scope for future improvements and enhancements to make the system more secure, scalable, and reliable.

#### 10.1 Limitations of the System

Although the system performs its intended functions, the following limitations are observed:

##### 1. Limited Security Features

- The system uses basic authentication (username and password).
- Advanced security techniques like encryption and multi-factor authentication are not implemented.

##### 2. Internet Dependency

- The system requires a stable internet connection.
- Users cannot vote offline.

##### 3. Scalability Issues

- The system is suitable for small-scale usage.
- It may face performance issues with a large number of users simultaneously.

##### 4. No Biometric Verification

- User identity verification is limited.
- No biometric or Aadhaar-based authentication is implemented.

##### 5. Risk of Cyber Attacks

- Vulnerable to hacking if deployed without strong security measures.
- Needs better protection against SQL Injection and data breaches.

##### 6. Limited User Interface Design

- UI is simple and basic.
- Can be improved for better user experience.

##### 7. No Real-Time Monitoring

- No live tracking of voting statistics.
- Results are only visible after processing.

## 10.2 Future Scope of the System

The Online Voting System has a wide scope for future development:

### 1. Advanced Security Implementation

- Use encryption techniques (SSL/TLS)
- Implement OTP-based login
- Multi-factor authentication

### 2. Biometric Verification

- Integration with fingerprint or facial recognition
- Aadhaar-based authentication system

### 3. Cloud Deployment

- Deploy system on cloud platforms (AWS, Azure)
- Improve scalability and availability

### 4. Mobile Application Development

- Android/iOS app for easy access
- Better user engagement

### 5. Real-Time Results and Analytics

- Live vote counting system
- Graphical dashboards for results

### 6. Blockchain Integration

- Use blockchain technology for transparency
- Secure and tamper-proof voting records

### 7. Multi-language Support

- Support different regional languages
- Improve accessibility for all users

## 10.4 Enhancements

Enhancements focus on improving system performance and usability.

### 1. User Interface Improvement

- Modern UI design using advanced CSS frameworks
- Better navigation and user experience

### 2. Performance Optimization

- Faster database queries
- Load balancing for handling large traffic

### 3. Security Enhancements

- Password encryption (hashing)
- Secure session management
- Protection against SQL Injection

### 4. Notification System

- Email/SMS confirmation after voting
- Alerts for registration and results

### 5. Admin Panel Improvement

- Advanced dashboard for admin
- Manage users, candidates, and results easily

### 6. Audit and Logging System

- Maintain logs of user activities
- Helps in tracking and security monitoring

## Chapter 11: References & Bibliography

### Introduction

References and Bibliography provide the list of all sources that were used during the development of the Online Voting System.

These sources include books, websites, research papers, and documentation that helped in understanding technologies such as HTML, CSS, JavaScript, Python, Flask, and MySQL.

### 11.1 References

The following references were used for developing the project:

#### Websites:

- (Official Python Documentation)  
<https://docs.python.org/3/>
- Flask Official Website  
<https://flask.palletsprojects.com/>
- W3Schools (HTML, CSS, JavaScript tutorials)  
<https://www.w3schools.com/>
- GeeksforGeeks (Programming concepts and examples)  
<https://www.geeksforgeeks.org/>
- Stack Overflow (Error solving and debugging)  
<https://stackoverflow.com/>

#### Tutorials and Resources:

- YouTube tutorials on Flask and Web Development
- Online coding platforms and blogs
- Free documentation and articles

### 11.2 Bibliography

Bibliography includes all sources referred to for knowledge and understanding, even if not directly cited.

#### General Resources:

- Articles on Online Voting Systems
- Research papers on Digital Voting Security
- Blogs related to Python and Web Development
- Documentation of MySQL Database

### 11.3 Importance of References

- Helps in validating the project work
- Provides reliability and authenticity
- Avoids plagiarism
- Helps readers explore more about the topic
- Shows proper research and understanding

### Conclusion

The references and bibliography section highlights all the sources that contributed to the successful development of the Online Voting System. These sources provided valuable guidance in understanding concepts, implementing technologies, and solving problems during the project development process.





